CSE D 501

Regularization and Dimensionality Reduction

Taylor Kessler Faulkner

University of Washington

October 8, 2025

Today’s Class

- Intro to classification
- Break
- Logistic Regression, Classification metrics
- Break
- Multinomial classification

Intro to Classification

Intro to Classification

- Regression: Given input x, predict continuous value $y \in \mathbb{R}$
- Classification: Given input x, predict discrete value $y \in \mathcal{Y}$

x

y

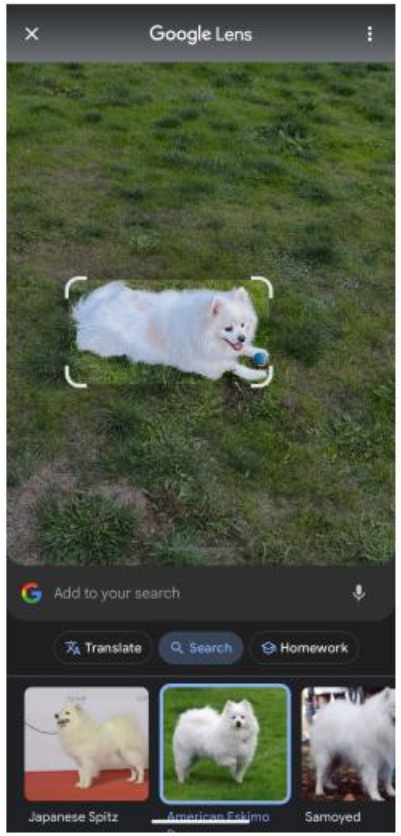
Regression: 62 degrees F
Out of possible degrees F

Classification: rainy
Out of [sunny, cloudy, rainy]

Intro to Classification

- Classification is everywhere!

x



y

Enter text:

One, two,

One , two ,

3198 11 734 11

Prediction

#	probs	next token ID	predicted next token
0	39.71%	1115	three
1	16.97%	290	and
2	7.55%	734	two
3	3.76%	1440	four
4	2.76%	393	or
5	2.18%	1936	five
6	1.57%	530	one
7	1.43%	345	you
8	1.15%	257	a
9	0.84%	3598	seven

https://huggingface.co/blog/alonsosilva/nexttokenprediction

Intro to Classification

- Learn f: $\mathcal{X} \rightarrow \mathcal{Y}$
  - $\mathcal{X} \in \mathbb{R}^D$ is the set of features
    - Still D total features
  - $\mathcal{Y} = \{1, \ldots, k\}$ is the set of target classes
    - Now a discrete set of possibilities instead of a continuous value

- Binary classification: predict 0 and 1 ($k = 2$)

- Multi-class classification: predict one of $k > 2$ classes

- We'll start by focusing on **binary classification**

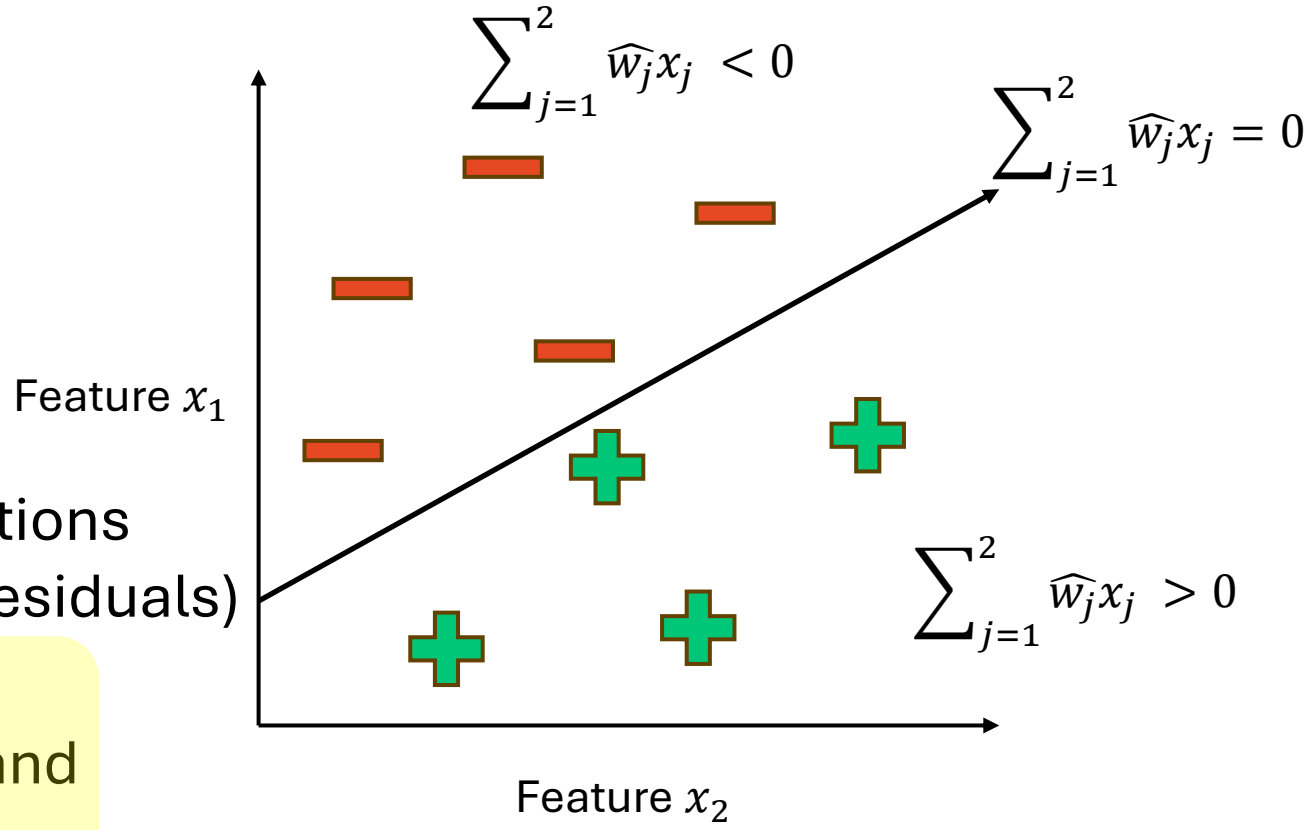
Intro to Classification

- Learn $f: \mathcal{X} \rightarrow \mathcal{Y}$
  - $\mathcal{X} \in \mathbb{R}^D$ is the set of features
    - Still D total features
  - $\mathcal{Y} = \{1, \ldots, k\}$ is the set of target classes
    - Now a discrete set of possibilities instead of a continuous value

- Consider Linear Regression: could we turn this regression algorithm into a binary classification algorithm?

Intro to Classification

- Initial idea: take our Linear Regression Model and output
  - +1 if $f(x) \geq 0$
  - -1 if $f(x) < 0$
- So $\hat{y} = \begin{cases} +1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x) \geq 0 \\ -1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x) < 0 \end{cases}$
- Some problems:
  - Sensitive to outliers
  - Linear Regression statistical assumptions are often violated (e.g. normality of residuals)
  - $\sum_{j=1}^{D} \widehat{w_j} h_j(x)$ is unbounded, so can't be interpreted as a probability and MSE doesn't work as intended



Intro to Classification

- What about modifying MSE to create a new **loss function?**
  - Loss function: measurement of model misfit
  - **0-1 loss function**: $\ell(f(x), y) = \frac{1}{n}\sum_{i=1}^{n} \delta_{f(x_i)\neq y_i}$, $where\ \delta_{f(x_i)\neq y_i} \begin{cases} 1\ if\ f(x_i) \neq y \\ 0\ if\ f(x_i) = y \end{cases}$
  - Also sometimes written as $\ell(f(x), y) = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{f(x) \neq y_i\}$
  - Effectively, the number of all instances of incorrect classifications over all classifications

- This loss function is not differentiable or continuous, so we can't use Least Squares or gradient descent

Perceptron Linear Classifier

- The **perceptron** algorithm uses an **update rule** rather than gradient descent to find good settings for w
- In this problem, a **perceptron** is an algorithm for binary classification that uses a prediction function called a **step function**

$$\hat{y} = \begin{cases} +1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x) \geq 0 \\ -1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x) < 0 \end{cases}$$

- This is just our method for converting linear regression to binary classification!
- You may have also heard the term perceptron used when talking about neural networks/deep learning
  - We'll cover this later in the quarter, but keep in mind that the **step function** is not the only function you can use with the output of $\sum_{j=1}^{D} \widehat{w_j} h_j(x)$
  - More complex functions (and combining many perceptron outputs) can create complex behaviors

Perceptron Linear Classifier

- The **perceptron** algorithm learns weights by the following pseudocode:

1. Initialize all weights/parameters w to 0 (or small random values)
2. For each $x_i \in \boldsymbol{x}$
    1. Classify x_i: $\widehat{y_i} = \begin{cases} +1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x_i) \geq 0 \\ -1 \; if \sum_{j=1}^{D} \widehat{w_j} h_j(x_i) < 0 \end{cases}$
    2. If $\widehat{y_i} = y_i$:
        1. Do nothing!
    3. If $\widehat{y_i} \neq y_i$:
        1. Modify the weights using an **update rule**
3. Repeat until convergence (all inputs are classified correctly)

Perceptron Linear Classifier Update Rule

- The **perceptron** update rule is

$$w_j^{(t+1)} = w_j^{(t)} + \eta(y_i - \hat{y}_i)x_i[j]$$

  - w_j:the weight of feature j
  - $x_i[j]$: the value of feature j for x_i
  - η: the step size

If $y_i = \hat{y}_i$: 0
If $y_i \neq \hat{y}_i$: -2 or +2, depending on $y_i = -1$ or 1

- This can also be written in matrix form

$$w^{(t+1)} = w^{(t)} + \eta(y - \hat{y})x$$

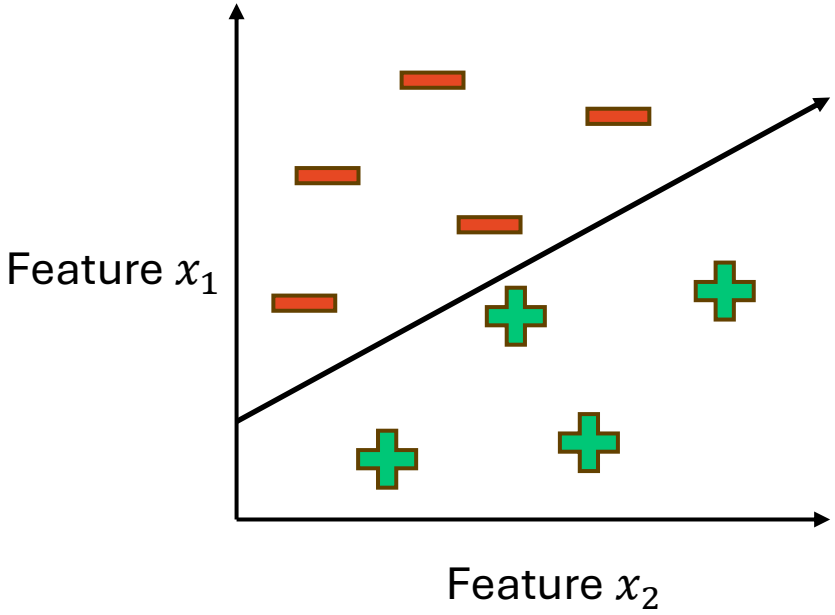
- This should look similar to the gradient descent update rule

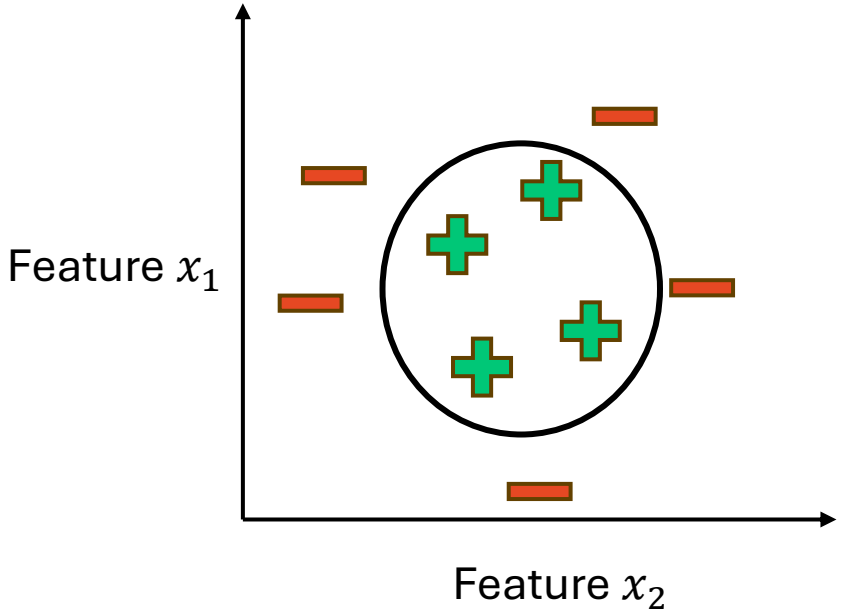
$$w^{(t+1)} = w^{(t)} - \eta\nabla \mathrm{L}(\mathrm{w})$$

- Just like in gradient descent, if η is too small, the algorithm will be very slow, and if η is too large, it may overshoot
  - In this algorithm, η is often set to 1

Perceptron Linear Classifier Update Rule

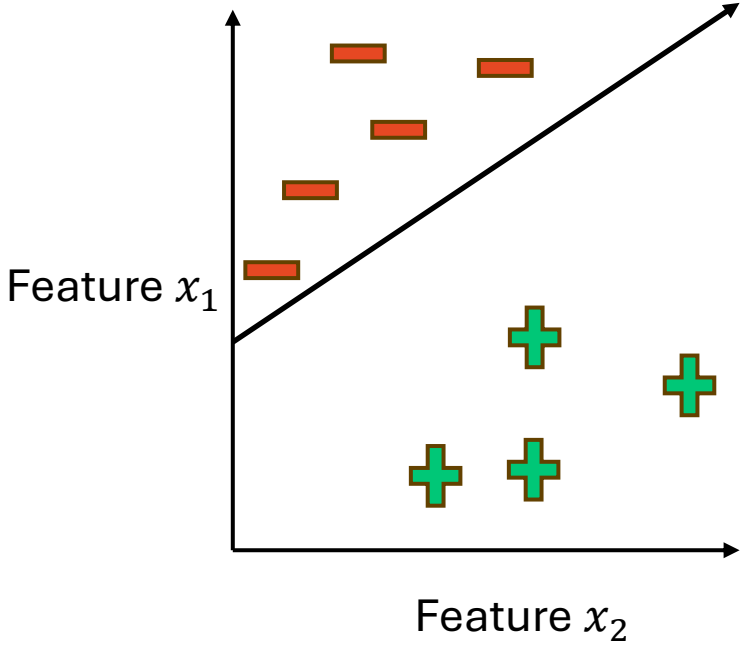
- The **perceptron** is guaranteed to converge if the data is **linearly separable**
  - **Linearly separable**: in 2D, there exists a line that will separate the two classes
    - In 3D, there exists a plane
    - In nD, there exists a hyperplane
    - A line is defined by $wx + b = 0$, a hyperplane is defined by $w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = 0$

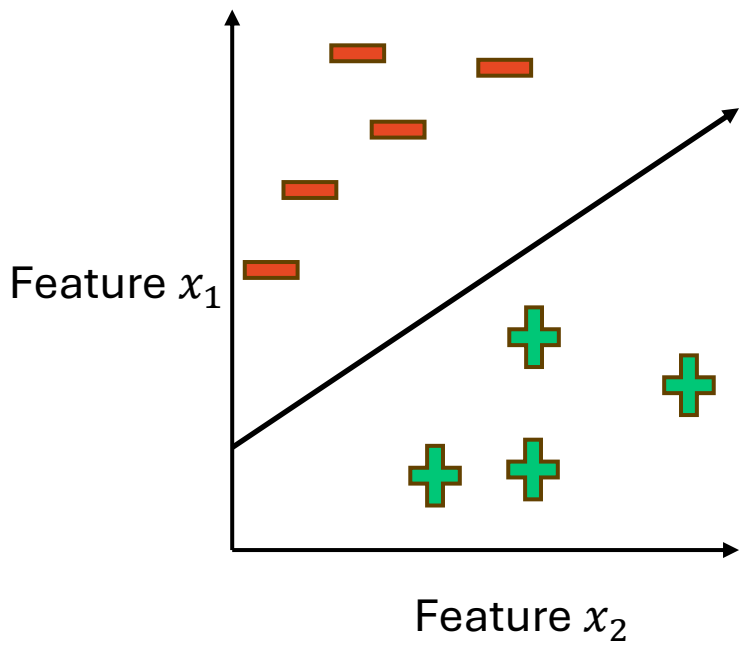




Perceptron Linear Classifier Update Rule

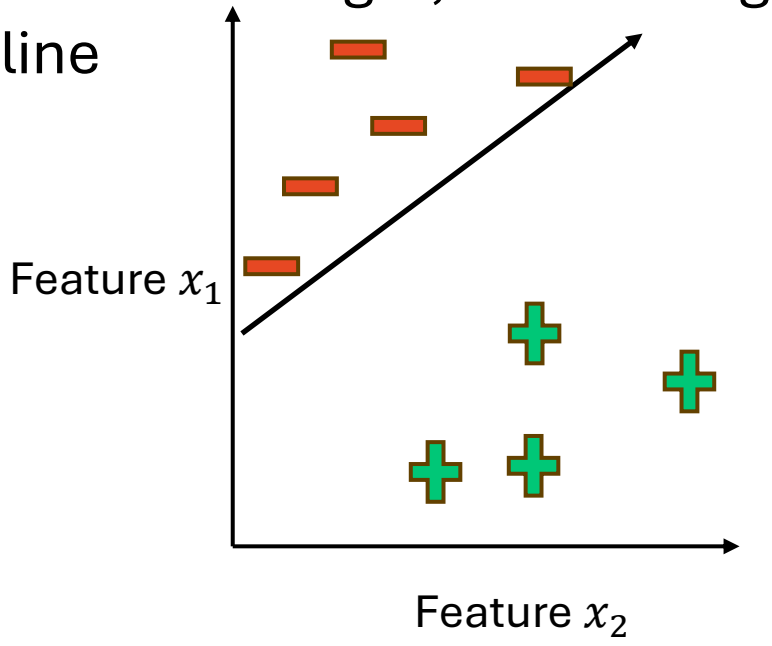
- The **perceptron** is guaranteed to converge if the data is **linearly separable**
- If the data is not linearly separable, the algorithm will not converge
- Even if the data is linearly separable, the algorithm tends to converge slowly
- Additionally, the separating line might be very close to the training data
- Question: which one of the below plots is more generalizable to future data?

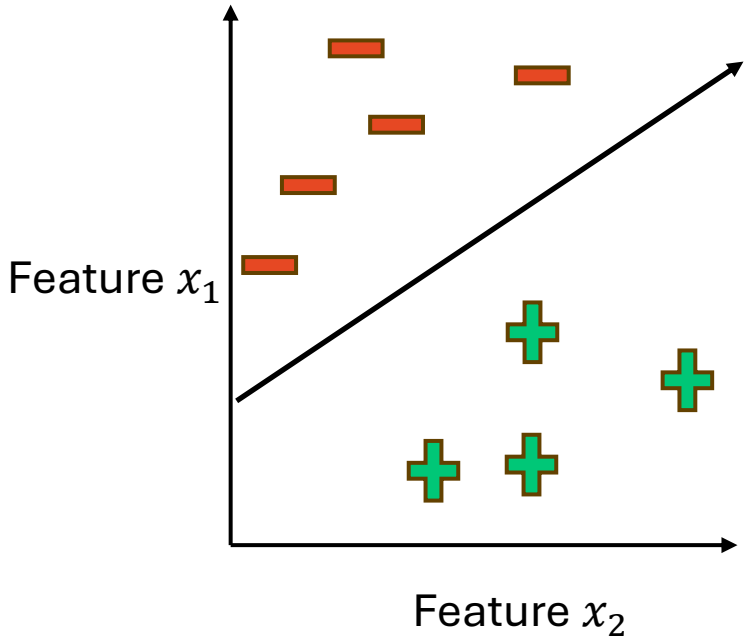




Perceptron Linear Classifier Update Rule

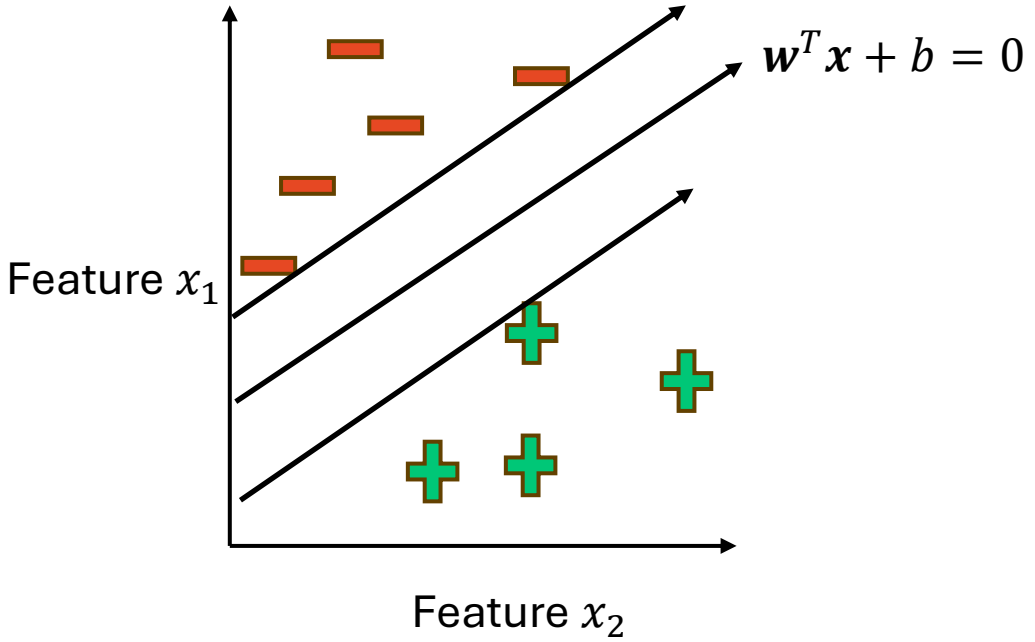
- The **perceptron** is guaranteed to converge if the data is **linearly separable**
- If the data is not linearly separable, the algorithm will not converge
- Even if the data is linearly separable, the algorithm tends to converge slowly
- Additionally, the separating line might be very close to the training data
- Question: which one of the below plots is more generalizable to future data?
- Answer: the one on the right, as new negative examples that come in might cross over the left line





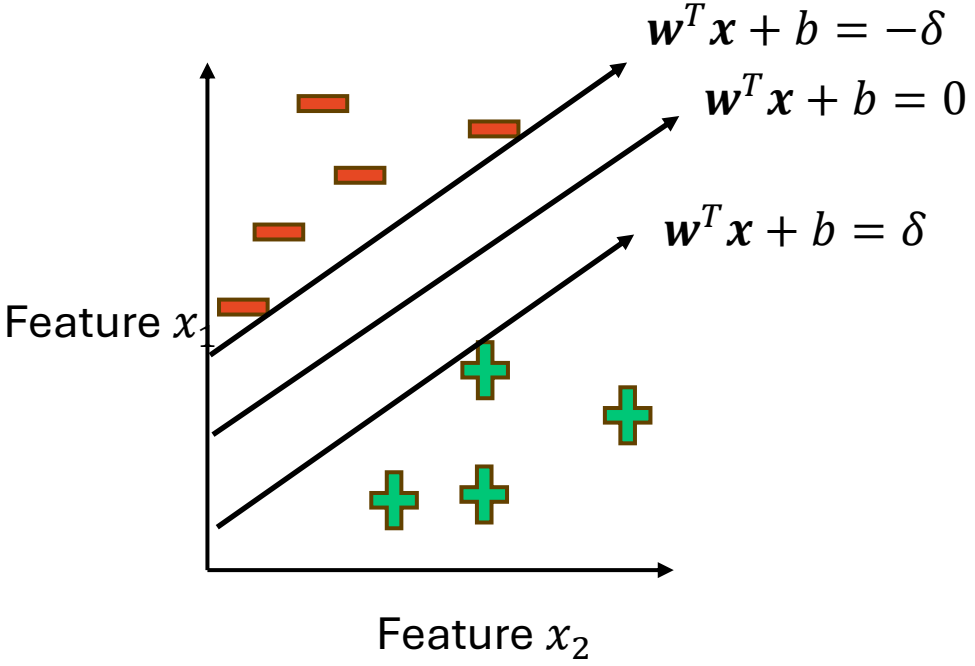
Support Vector Machines (SVMs)

- How do we find the setting of w corresponding to a hyperplane ($\boldsymbol{w}^T\boldsymbol{x} + b = 0$) that maximizes the distance from both classes?
- We can define two more hyperplanes right at the boundary of the positive and negative classes (parallel to the currently guessed hyperplane)
- These hyperplanes are called **support vectors**



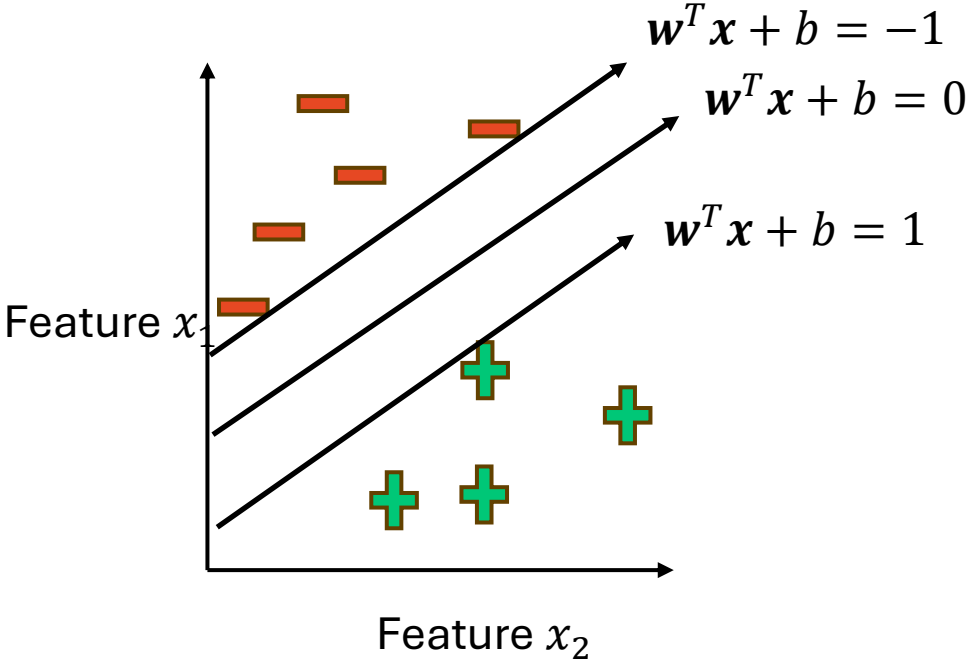
Support Vector Machines (SVMs)

- The **support vectors** will be of the form $\boldsymbol{w}^T\boldsymbol{x} + b = \delta$ and $\boldsymbol{w}^T\boldsymbol{x} + b = -\delta$ when $\boldsymbol{w}^T\boldsymbol{x} + b = 0$ is in the optimal location (halfway between the two support vectors)
- We can set $\delta = 1$ WLOG (without loss of generality)
  - This means that we can set $\delta = 1$ without changing the initial problem, because no matter what δ is, we can scale it and w and b can be scaled to adjust
  - Like how we can scale our features for linear regression



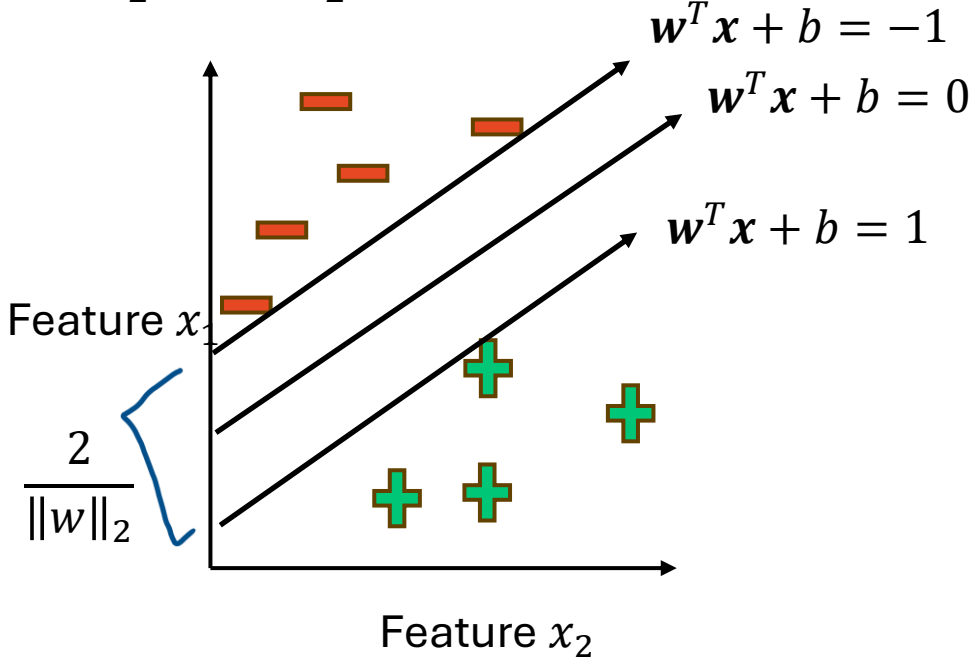
Support Vector Machines (SVMs)

- Now that we have defined our **support vectors** as $\boldsymbol{w}^T\boldsymbol{x} + b = 1$ and $\boldsymbol{w}^T\boldsymbol{x} + b = -1$, we can find the **margin** (distance between the two support vectors)
- The support vectors written as hyperplanes are $\boldsymbol{w}^T\boldsymbol{x} + (b - 1) = 0$ and $\boldsymbol{w}^T\boldsymbol{x} + (b + 1) = 0$



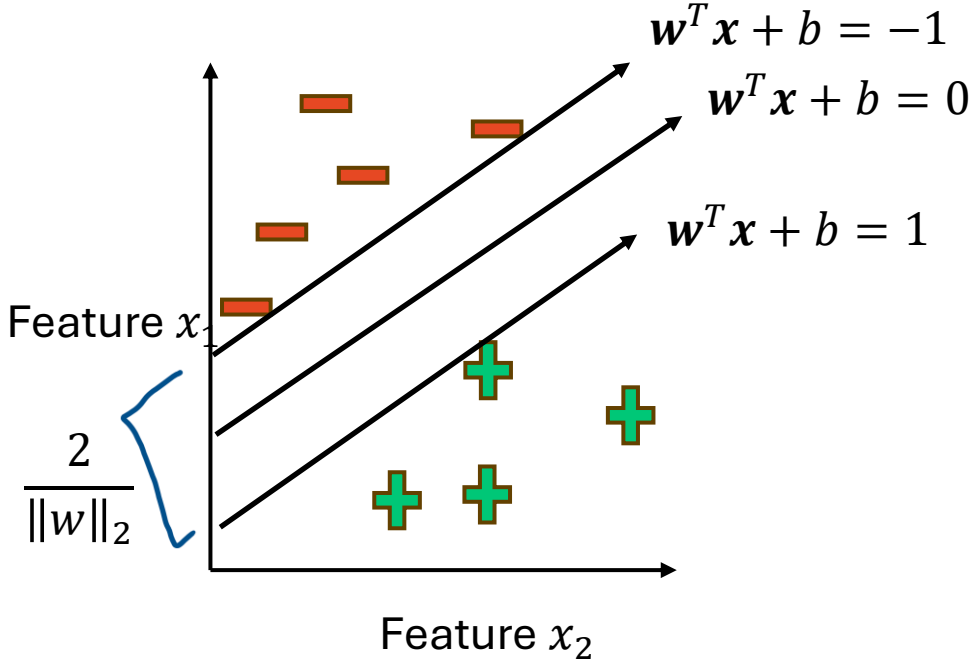
Support Vector Machines (SVMs)

- Support vector hyperplanes: $\boldsymbol{w}^T\boldsymbol{x} + (b - 1) = 0$ and $\boldsymbol{w}^T\boldsymbol{x} + (b + 1) = 0$
- The displacement from the origin (0,0) to a general hyperplane is defined as $\frac{b}{\|w\|_2}$
- The distance to the hyperplanes from the origin are $\frac{b+1}{\|w\|_2}$ and $\frac{b-1}{\|w\|_2}$, and their distance from each other is $\frac{b+1}{\|w\|_2} - \frac{b-1}{\|w\|_2} = \frac{2}{\|w\|_2}$



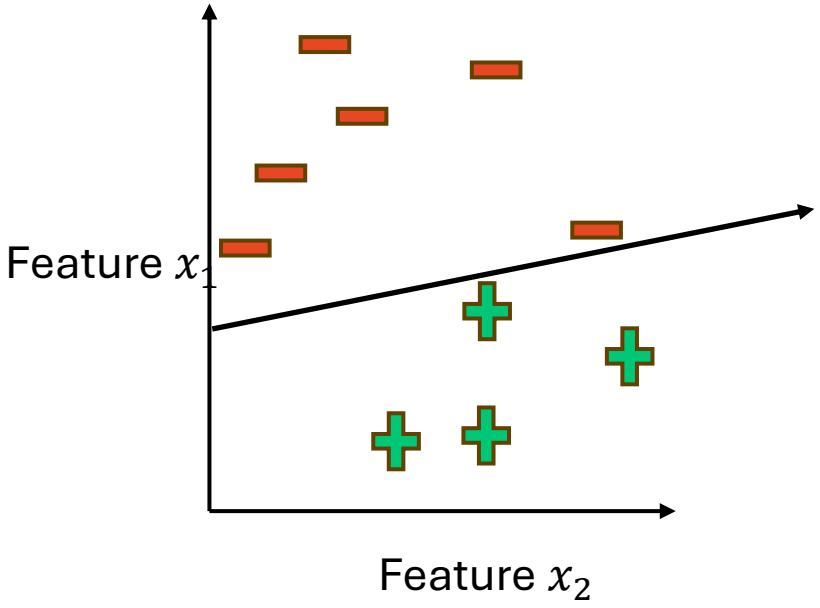
Support Vector Machines (SVMs)

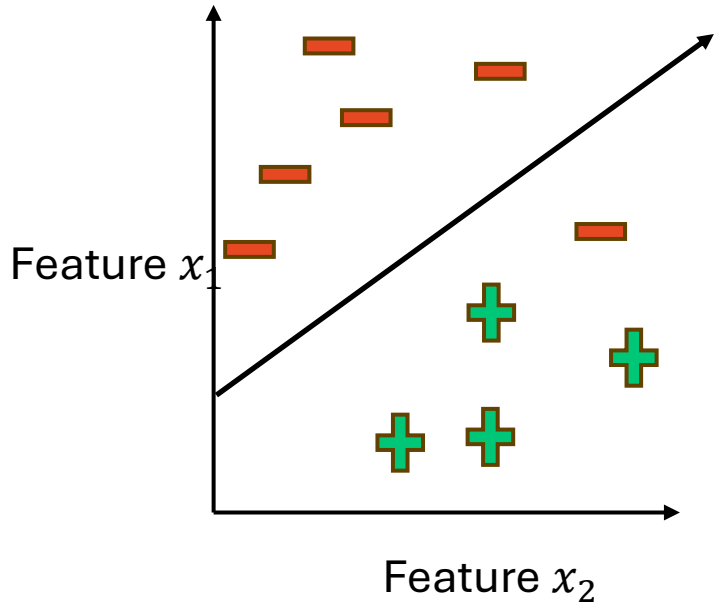
- Learning the right w for the SVM is an optimization problem:
  - $\max_w \frac{2}{\|w\|_2}$ subject to constraints: $\mathbf{w}^{\mathrm{T}}\mathrm{x_i} + \mathrm{b}$ is $\begin{cases} \geq 1 \; if \; y_i = +1 \\ \leq -1 \; if \; y_i = -1 \end{cases}$
  - Or $\min_w \frac{1}{2}\|w\|_2^2$ subject to constraints: $y_i(\boldsymbol{w}^T x_i + b) \geq 1$



Support Vector Machines (SVMs)

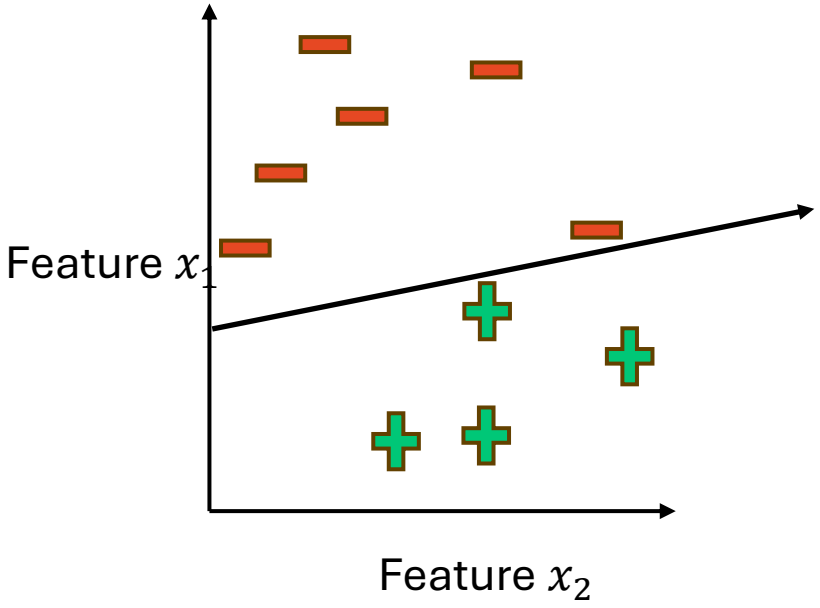
- What about data that is linearly separable, but potentially has outliers that should be ignored?
- Question: is the hyperplane on the left or right better?

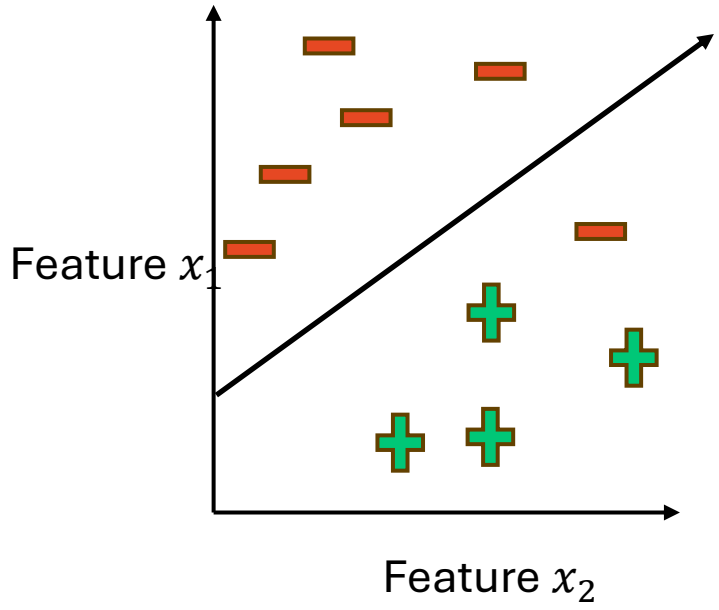




Support Vector Machines (SVMs)

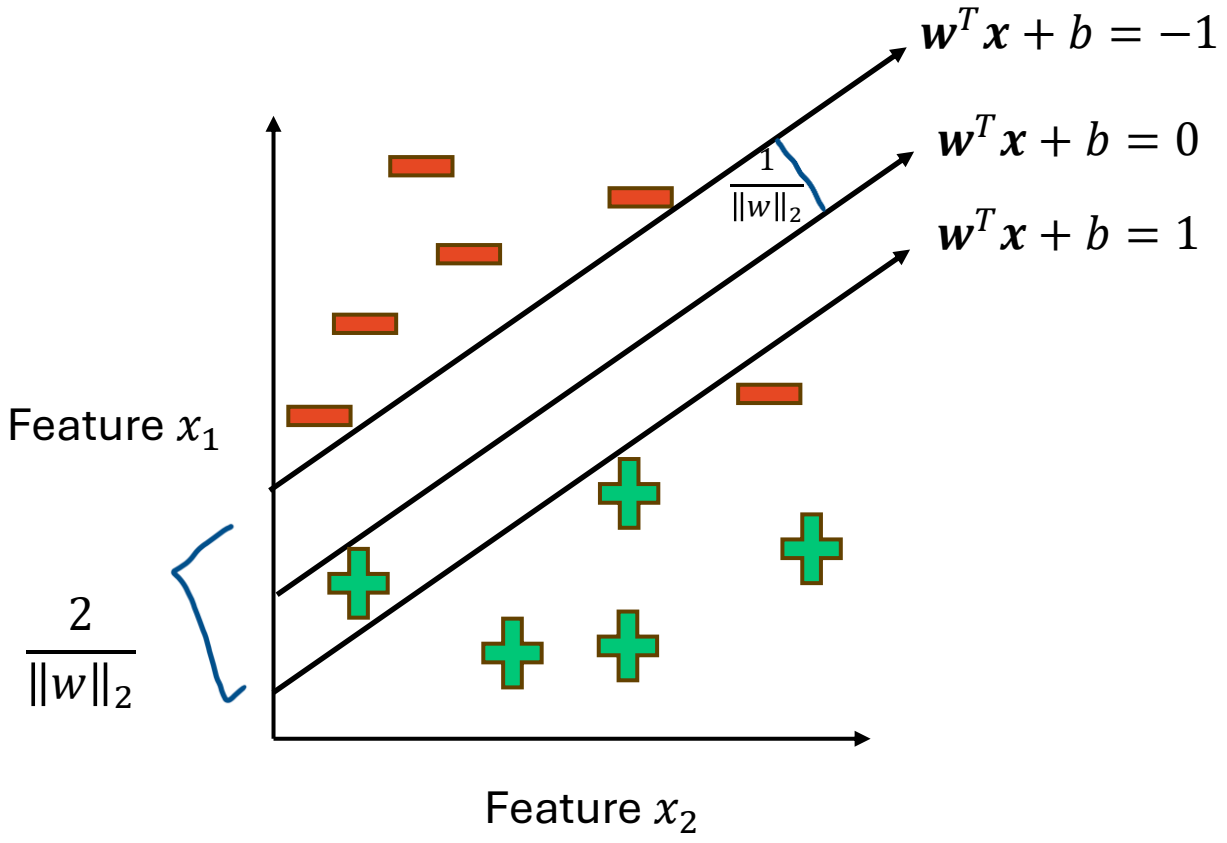
- What about data that is linearly separable, but potentially has outliers that should be ignored?
- Question: is the hyperplane on the left or right better?
- Answer: it depends on the problem; is it better to classify all training data correctly, or find a hyperplane with a large margin that still classifies most of the data correctly?





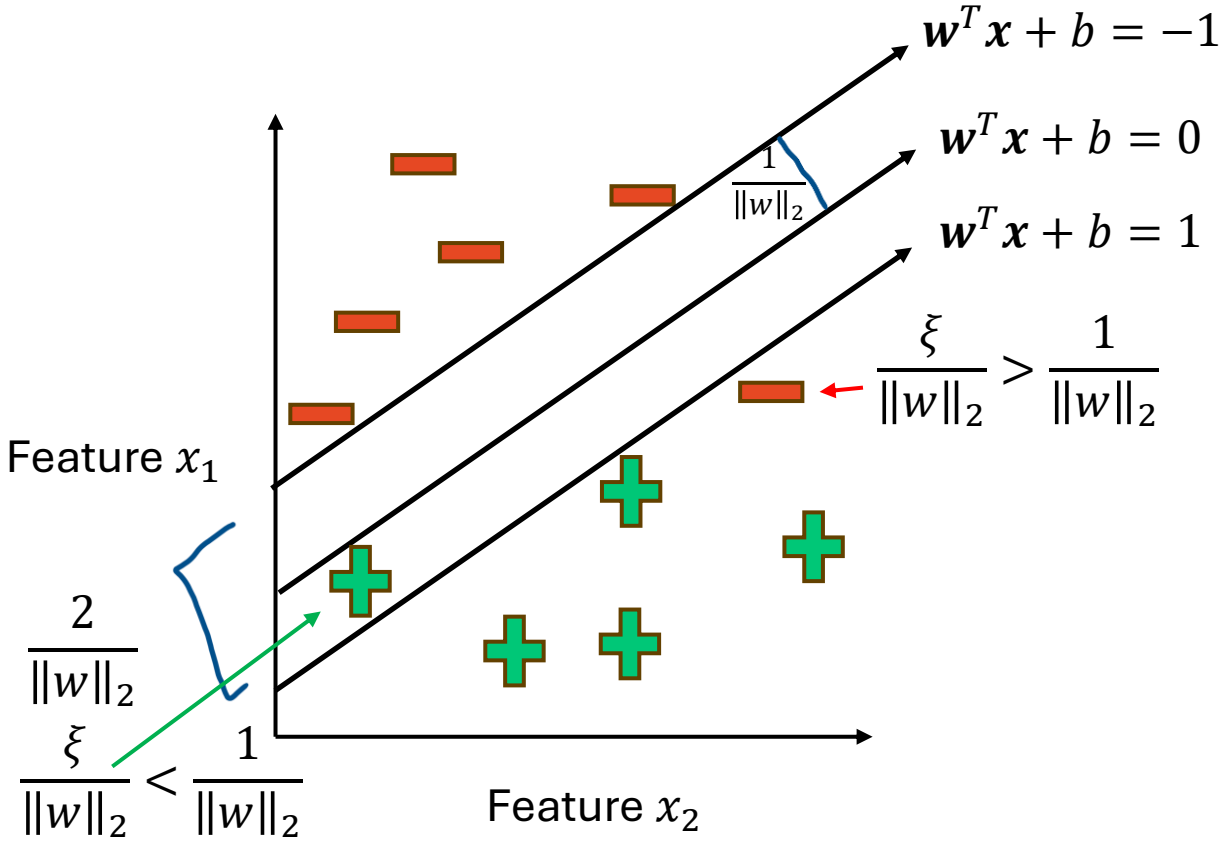
Support Vector Machines (SVMs)

- We can introduce **slack** variables to allow for misclassification
- If we look at the distance from the chosen hyperplane of our points, we can define a variable $\xi \geq 0$



Support Vector Machines (SVMs)

- For $0 < \xi \leq 1$, a point is between the margin and the correct side of the hyperplane
  - **Margin violation**
- For $\xi > 1$, a point is on the wrong side of the hyperplane
  - **Misclassified**



Support Vector Machines (SVMs)

- Now we can optimize:
  - $\min_{w,\xi_i \in \mathbb{R}^+} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}\xi_i$ subject to constraints: $y_i(\boldsymbol{w}^T x_i + b) \geq 1 - \xi_i$

  - C is a regularization parameter
    - Small C allows constraints to be easily ignored and increases the margin
    - Large C makes constraints harder to ignore and decreases the margin
    - $C = \infty$ enforces all constraints

Support Vector Machines (SVMs)

- Instead of solving a constrained problem, we can rewrite the formula to optimize as **hinge loss**
- We start here:

$$\min_{w,\xi_i \in \mathbb{R}^+} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}\xi_i \text{ subject to constraints: } y_i(\boldsymbol{w}^T x_i + b) \geq 1 - \xi_i$$

- The constraint $y_i(\boldsymbol{w}^T x_i + b) \geq 1 - \xi_i$ can be written as

$$y_i(f(x_i)) \geq 1 - \xi_i$$

- We enforce $\xi \geq 0$, so we can write the above as

$$\xi_i = \max(0, 1 - y_i f(x_i))$$

- So, we can rewrite the entire formula as

$$\min_{w} \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n}\max(0, 1 - y_i f(x_i))$$

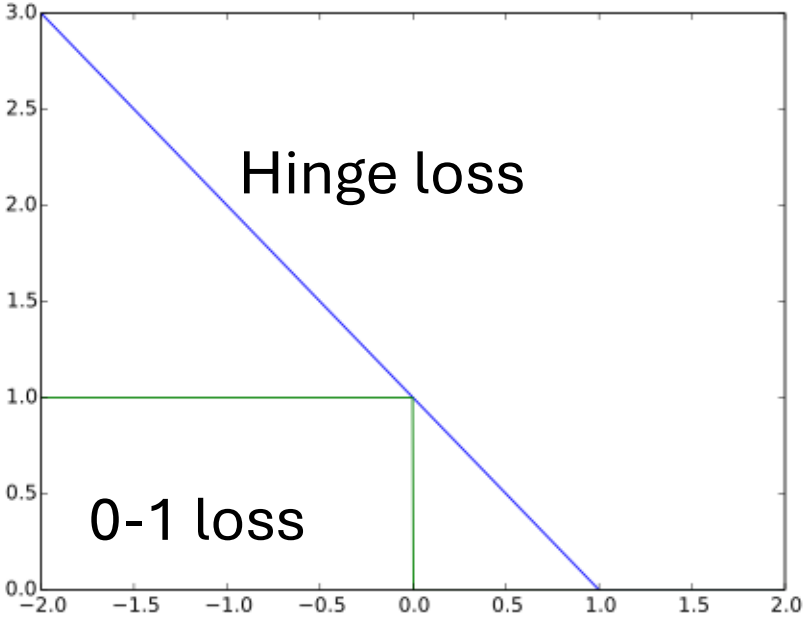
Regularization Hinge loss

Support Vector Machines (SVMs)

- Now the points are in three categories:
  - $y_i f(x_i) > 1$: outside margin on the correct side, no contribution to loss
  - $y_i f(x_i) = 1$: on the margin line, no contribution to loss
  - $y_i f(x_i) < 1$: point violates margin constraint, and contributes to loss

$$\min_w \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i f(x_i))$$

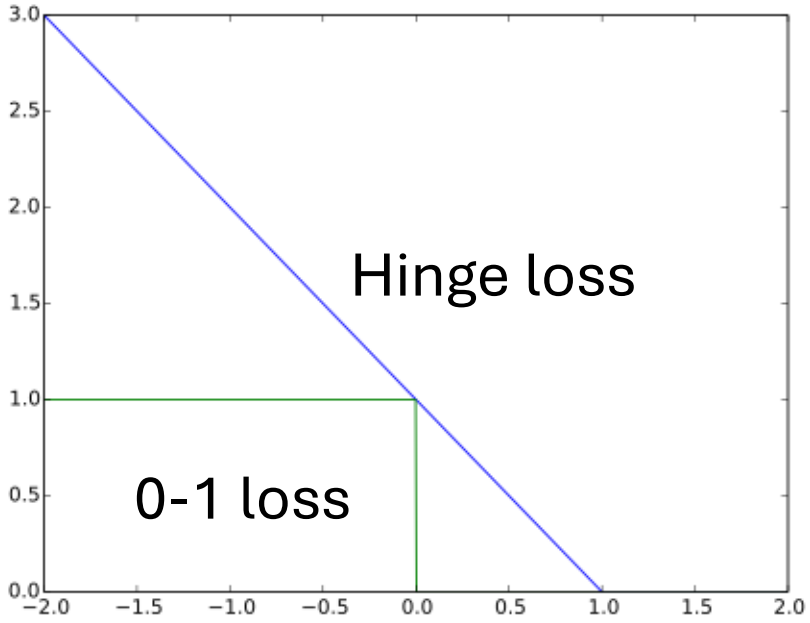
- Hinge loss is an approximation of 0-1 loss that maxes at 0 but can be infinitely negative



Support Vector Machines (SVMs)

- Can we use gradient descent on Hinge loss?
  - To find a global optimum using gradient descent, functions must be **convex**
  - To compute gradients, functions must be **differentiable**
  - Hinge loss is convex but not differentiable when $y_i f(x_i) = 1$

- That's ok; we can take **subgradients** of hinge loss!
  - The gradient when $y_i f(x_i) > 1$ is 0
  - The gradient when $y_i f(x_i) < 1$ is $-y_i x_i$



Support Vector Machines (SVMs)

- We can take **subgradients** of hinge loss!
  - The gradient when $y_i f(x_i) > 1$ is 0
  - The gradient when $y_i f(x_i) < 1$ is $-y_i x_i$
- Now we can take the gradient of our SVM equation, and apply two different gradients: 0 when $y_i f(x_i) \geq 1$, and $-y_i x_i$ otherwise
  - Just an if statement in the gradient descent loop
- I'll leave it as an exercise to try taking the gradient of the whole formula:

$$\min_w \frac{1}{2}\|w\|_2^2 + C\sum_{i=1}^{n} \max(0, 1 - y_i f(x_i))$$

Support Vector Machines (SVMs)

- That was a lot of math
- SVMs get even more math-heavy if you want to make them work for non-linearly separable data
  - May or may not get to that this quarter
- We can also update Linear Regression in another (less math-heavy way)
- We'll look at that method after a break!

Break

Get some water, stretch your legs, chat, be back in 10

Logistic Regression

- What was our initial problem with using Linear Regression directly as a classifier?
  - $\sum_{j=1}^{D} \widehat{w_j} h_j(x)$ is unbounded, so can't be interpreted as a probability and MSE doesn't work as intended

- What if we modify the Linear Regression model so that output is always bounded between 0 and 1?

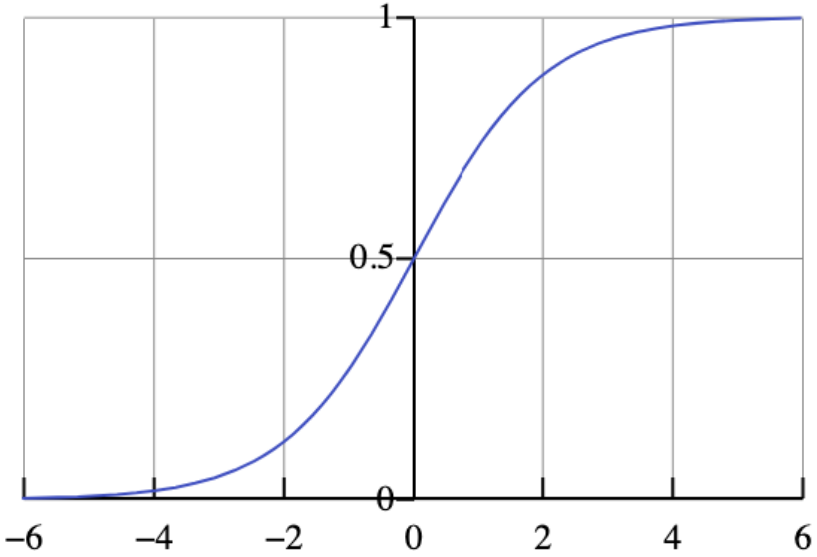
Logistic Regression

- There is a useful mathematical function that bounds unbounded input to the range [0,1]: the **logistic function**

$$\sigma(z) = \frac{e^z}{1+e^z} = \frac{1}{1+e^{-z}}$$

- To map the output of Linear Regression to a bounded value, we can use

$$\frac{1}{1+e^{-\hat{y}}} = \frac{1}{1+e^{-\widehat{w}^T x_i}}$$



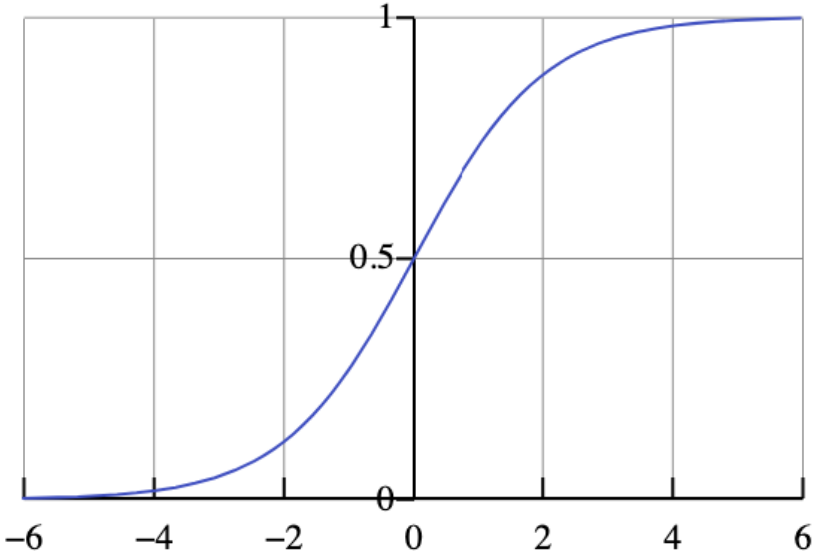
Logistic Regression

$$\frac{1}{1+e^{-\hat{y}}} = \frac{1}{1+e^{-\hat{w}^T x_i}}$$

- Now if $\hat{w}^T x_i > 0$, $0.5 < \frac{1}{1+e^{-\hat{w}^T x_i}} \leq 1$
- If $\hat{w}^T x_i < 0$, $0 \leq \frac{1}{1+e^{-\hat{w}^T x_i}} < 0.5$
- So we can make our decision boundary:
  - Predict positive if

$$\frac{1}{1+e^{-\hat{w}^T x_i}} > 0.5$$

  - Predict negative otherwise
- Now that our values are bounded between 0 and 1, how might we want to interpret the output?



Logistic Regression

$$\frac{1}{1+e^{-\hat{w}^T x_i}} > 0.5$$

- Now we can interpret the output as a probability!

$$P(y=+1|x) = \frac{e^{\hat{w}^T x_i}}{1+e^{\hat{w}^T x_i}} = \frac{1}{1+e^{-\hat{w}^T x_i}}$$

- If $\hat{w}^T x_i$ is positive, then $P(y=+1|x) > 0.5$
- If $\hat{w}^T x_i$ is negative, then $P(y=+1|x) < 0.5$
- What about $P(y=-1|x)$?

$$P(y=-1|x) = 1 - P(y=+1|x) = 1 - \frac{1}{1+e^{-\hat{w}^T x_i}} = \frac{e^{\hat{w}^T x_i}}{1+e^{-\hat{w}^T x_i}} = \frac{1}{1+e^{\hat{w}^T x_i}}$$

- Can interpret as probabilities: $P(y=+1|x) = 0.9$ is more likely that y is +1 than $P(y=+1|x) = 0.6$

Logistic Regression

- What loss function should we use for Logistic Regression?
- We want to find the setting of w that maximizes the probability of matching the provided labels in the dataset
  - Maximize the **likelihood** of matching the dataset
- The probability of any y_i being correctly predicted is

$$P(y_i|x_i) = \begin{cases} P(y_i = +1\,|x_i)\ if\ y_i = +1 \\ P(y_i = -1\,|x_i)\ if\ y_i = -1 \end{cases}$$

- To compute the likelihood of matching the dataset, we need to maximize the probability that each y_i is correctly predicted:

$$likelihood(w) = P(y_1|x_1) * P(y_2|x_2) * \cdots * P(y_n|x_n) = \prod_{i=1}^{n} P(y_i|x_i)$$

Logistic Regression

- We now have a likelihood metric that we want to maximize:

$$likelihood(w) = P(y_1|x_1) * P(y_2|x_2) * \cdots * P(y_n|x_n) = \prod_{i=1}^{n} P(y_i|x_i)$$

- Unfortunately there are a ton of matrix multiplications to do!
  - Matrix multiplication is much slower than matrix addition
  - Can we turn the product into a sum?
- Taking the log of a product can convert it into a sum! We'll use ln, natural log:

$$\ln(likelihood(w)) = \sum_{i=1}^{n} \ln(P(y_i|x_i))$$

Logistic Regression

- We've now derived **log-likelihood**:

$$\ln(likelihood(w)) = \sum_{i=1}^{n} \ln(P(y_i|x_i))$$

- This can be written out as

$$\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)$$

- The term on the left penalizes getting +1 examples wrong
- The term on the right penalizes getting -1 examples wrong
- One last addition: multiply by $\frac{1}{n}$ so we can compare across different dataset sizes

$$\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

Logistic Regression

- Sometimes binary classification is written as predicting 0 and 1, instead of -1 and 1
- When that is the case, this formula

$$\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

- Can be written as this

$$\frac{1}{n}\left(\sum_{i=1}^{n} (y_i * \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right)) + (1-y_i) * \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

- This last equation is very commonly referenced as the log likelihood formula
- But for terminology we're using later, predicting the two classes as -1 (negative) and +1 (positive) will make more sense

Logistic Regression

- We now have a differentiable convex function, great for gradient descent

$$\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

- But gradient descent **minimizes** values, and we want to **maximize** the log likelihood
- Two equivalent solutions:
  - Use **gradient ascent**
  - Use the log loss, the negative of log likelihood
  - This loss is known as **Binary Cross-Entropy Loss**

Gradient ascent (pseudocode)

$t = 0$

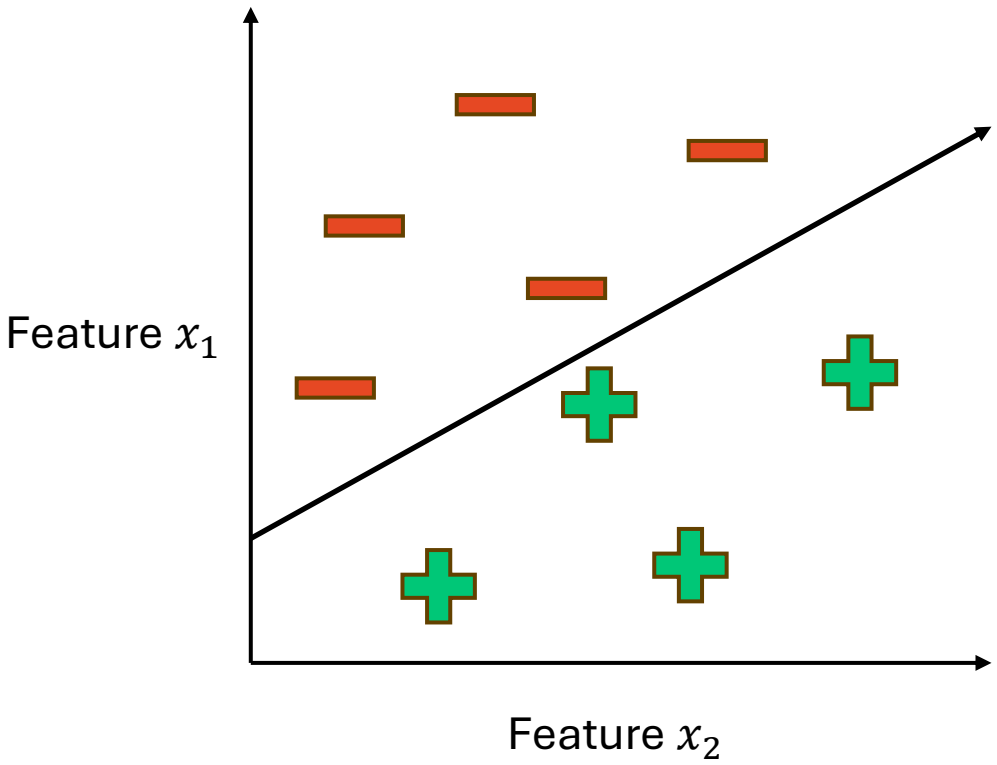
while not converged:

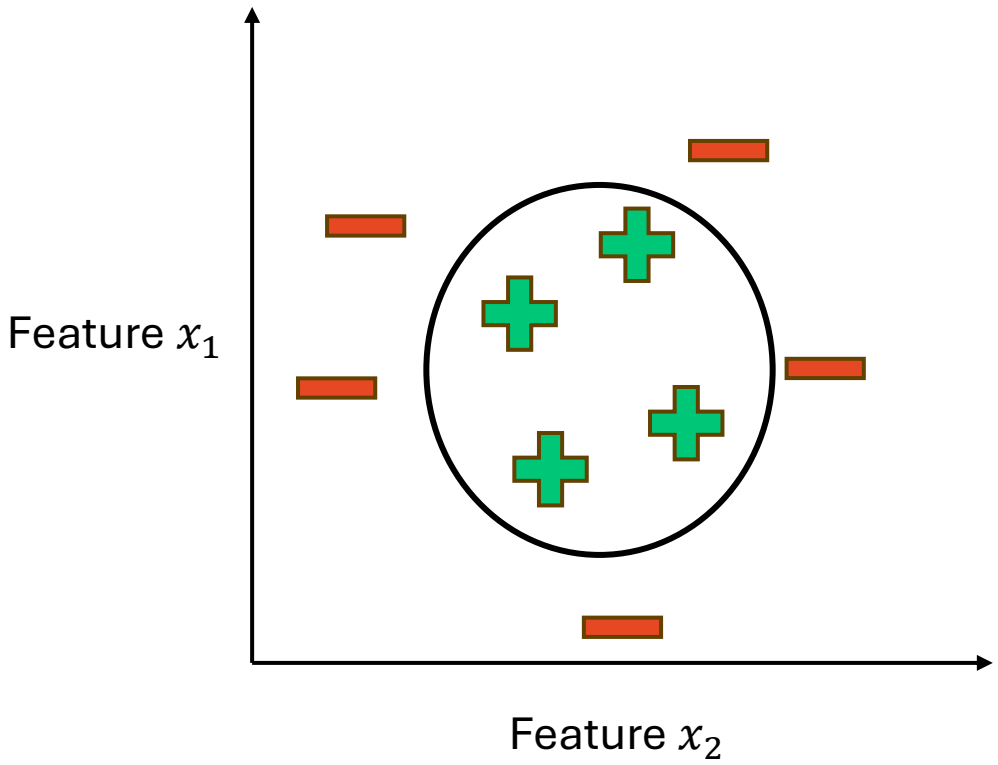
$$w^{(t+1)} = w^{(t)} + \eta \nabla \mathrm{L}(\mathrm{w})$$

$$-\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

Logistic Regression

- Not all data will be linearly separable!





Logistic Regression

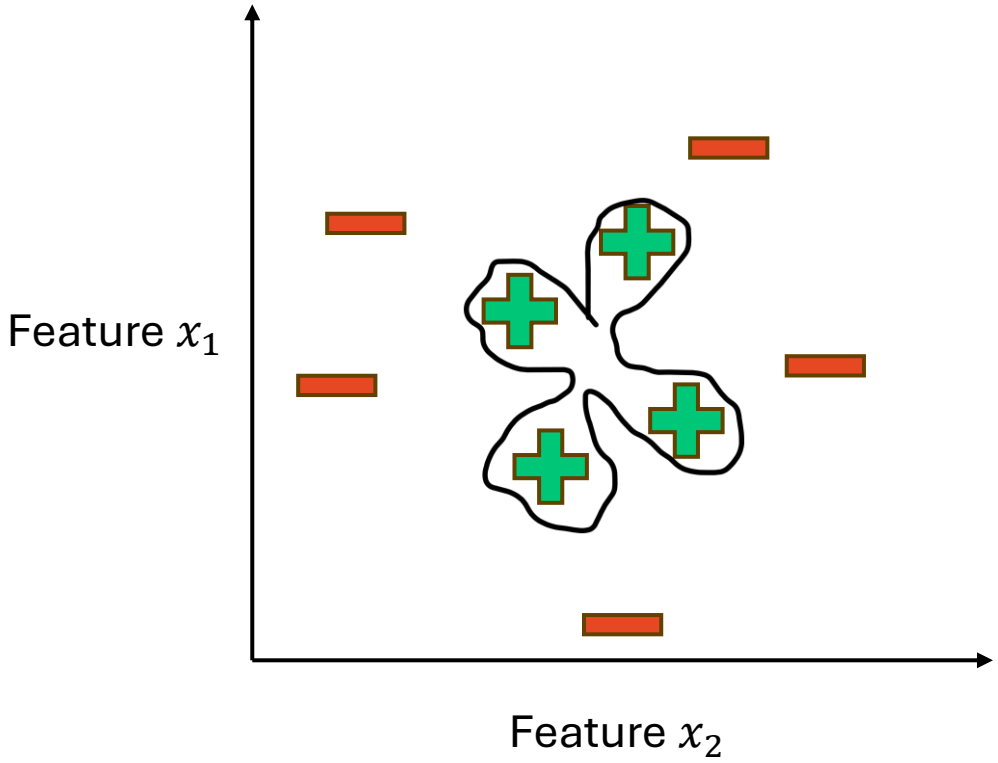
- Like Linear Regression, we can make our models more complex by:
  - Adding more features
  - Adding more complex features (higher degrees, etc)

- Instead of just using x_1, x_2, you could additionally use all degree 2 polynomial combinations
  - $x_1, x_2, x_1x_2, x_1^2, x_2^2$

- This is still finding a linear hyperplane, but reshaping the data
  - More on this next week

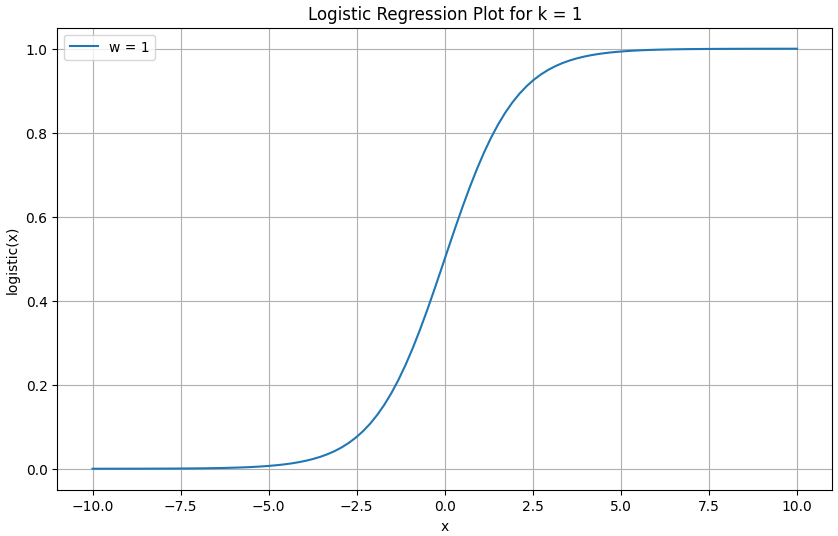
Logistic Regression

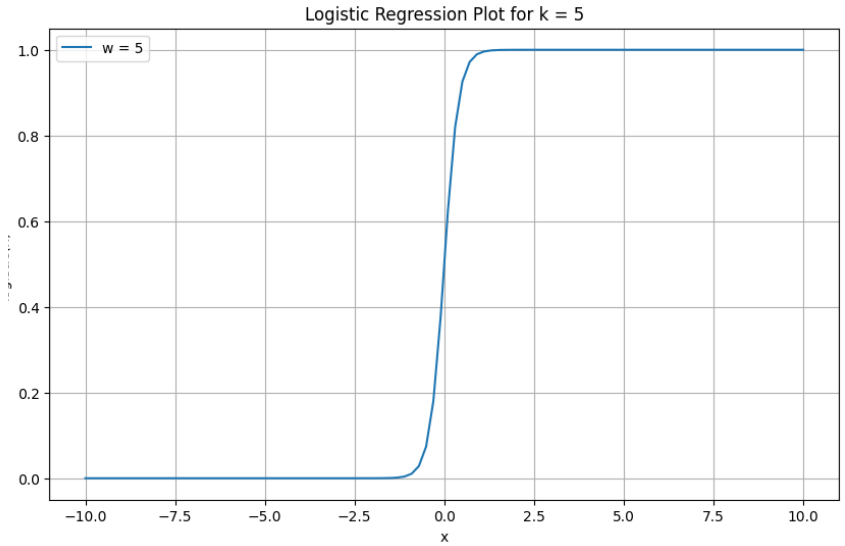
- But we can again make our model too complex and overfit the training data!

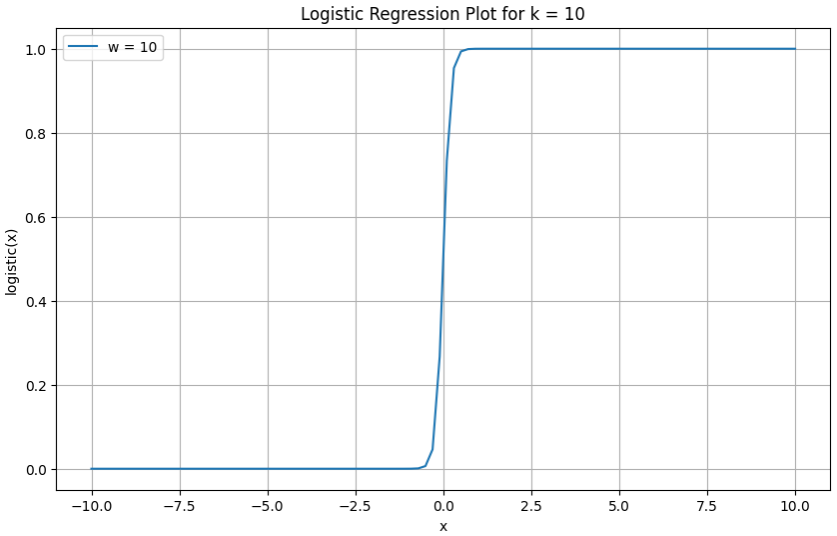


Logistic Regression

- As coefficients get larger, the logistic function changes
- Consider the following plots of the logistic function for $w_1 x_1$ for $w \in [1,5,10]$







- Because $w_1 x_1$ is increasing in magnitude, the classifier will output probabilities much closer to 0 or 1. The classifier will be overconfident!

Logistic Regression: Regularization

- As we learned last week, using L1 or L2 regularization (LASSO and Ridge) can help bring down coefficient magnitudes for regression
- We can do the same thing for Logistic Regression!
- Example for Ridge regularization:

$$\widehat{w} = argmin_w -\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right) + \lambda\|w\|_2^2$$

Evaluating Classifiers

Evaluating Classifiers

- How do we evaluate the performance of classifiers?
- With regression, we used MSE and R^2 to evaluate performance

- Initial idea: **accuracy**
  - The ratio of examples with a correct prediction
  - Mistakes: predicting +1 instead of -1, and vice versa
- Classification error

$$\frac{\#\ mistakes}{\#\ examples} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{\hat{y}_i \neq y_i\}$$

- Classification accuracy

$$\frac{\#\ correct}{\#\ examples} = \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}\{\hat{y}_i = y_i\}$$

Evaluating Classifiers

- For binary classification, we should at least beat random guessing (accuracy > 0.5)
- For multi-class classification (k classes), we want accuracy > $\frac{1}{k}$
- Question: does higher accuracy always mean a better classifier? Example: is a binary classifier with accuracy = 0.6 better than a classifier with accuracy = 0.1?

Evaluating Classifiers

- For binary classification, we should at least beat random guessing (accuracy > 0.5)
- For multi-class classification (k classes), we want accuracy > $\frac{1}{k}$
- Question: does higher accuracy always mean a better classifier? Example: is a binary classifier with accuracy = 0.6 better than a classifier with accuracy = 0.1?
- Answer: no: if you flip the 0.1 classifiers output, it is actually a good classifier with accuracy = 0.9

Majority Classes and Accuracy

- Imagine a very simple classifier for detecting spam emails: it always predicts spam
- This could actually have a very high accuracy, as most emails are in fact spam
- This represents an **unbalanced** dataset: when there are far more of some classes than others
- The very simple classifier above is called a **majority class classifier**, as it always predicts the majority class
  - This can have a high accuracy if there is a **class imbalance**
- So maybe getting above 50% accuracy doesn't tell us much if the dataset is imbalanced; if 80% of the data is the positive class, and we use a majority class classifier, we could get 80% accuracy
- You should always compare to a reasonable baseline by looking at your data
  - Often trying to beat random guessing or a majority classifier

Other metrics besides accuracy

- For binary classification, there are only two types of mistakes:
  - $\hat{y} = +1, y = -1$
  - $\hat{y} = -1, y = +1$
- We can represent these errors in a **confusion matrix**

		Predicted Label: Positive	Predicted Label: Negative
True Label	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

Discussion

Think/Pair/Share
Think for 5 minutes
Group discussion for 5-10 minutes
Share interesting examples with the class

Prompt: come up with a classification task in which the **false negative** or **false positive** rate might be more important/consequential than the overall classifier accuracy

- Describe what the positive and negative classes are, and which one is most important
- What are the possible consequences of maximizing accuracy over minimizing the false negative or positive rate in this task?

Other metrics besides accuracy

- We can represent (binary) accuracy as $\frac{TP+FN}{TP+TN+FP+FN}$
- We can also measure our true positive rate (**recall**): $\frac{TP}{TP+FN}$
- And our **precision**, all of the models positive classifications that were correct: $\frac{TP}{TP+FP}$

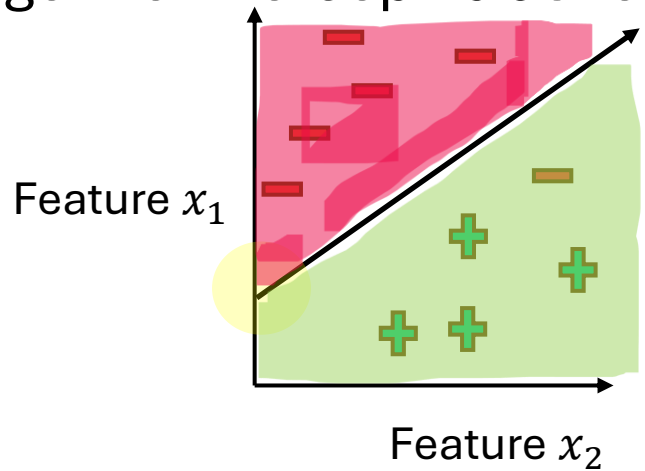
		Predicted Label: Positive	Predicted Label: Negative
True Label	Positive	True Positive (TP)	False Negative (FN)
	Negative	False Positive (FP)	True Negative (TN)

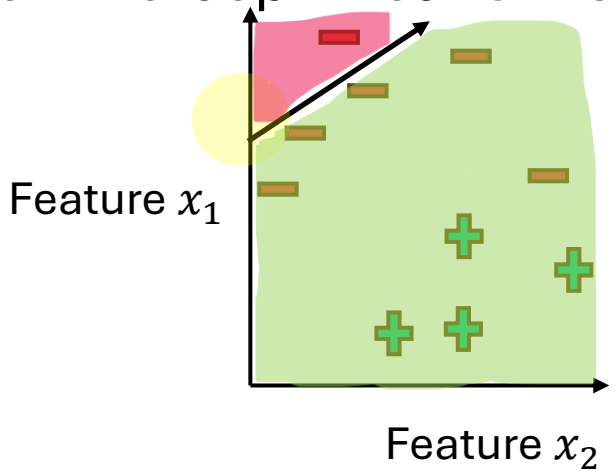
Other metrics besides accuracy

- Now we can represent (binary) accuracy as $\frac{TP+FN}{TP+TN+FP+FN}$
- We can also measure our true positive rate (**recall)**: $\frac{TP}{TP+FN}$
- And our **precision**, all of the models positive classifications that were correct: $\frac{TP}{TP+FP}$
- We can also use an **F1 score**: this score balances precision and recall to give a single evaluation score: $2 * \frac{Precision * Recall}{Precision+Recall}$

Optimizing for Precision vs Recall

- **Precision:** of the samples the model predicted positive, how many were actually positive?
- **Recall**: of the samples that are actually positive, how many did the model predict as positive?
- If we never want to make a false positive prediction, we could always predict negative
- If we never want to make a false negative prediction, we could always predict positive
- To balance, we can
  - Change the intercept/bias term (higher intercept means more positive predictions)





Optimizing for Precision vs Recall

- **Precision:** of the samples the model predicted positive, how many were actually positive?
- **Recall**: of the samples that are actually positive, how many did the model predict as positive?
- If we never want to make a false positive prediction, we could always predict negative
- If we never want to make a false negative prediction, we could always predict positive
- To balance, we can
  - Change the intercept/bias term (higher intercept means more positive predictions)
  - Change the threshold for predicting positive vs negative
    - Instead of $\widehat{y_i} = +1 \; if \; \frac{1}{1+e^{-\widehat{w}^T x_i}} > 0.5, \widehat{y_i} = -1 \; otherwise$
    - $\widehat{y_i} = +1 \; if \; \frac{1}{1+e^{-\widehat{w}^T x_i}} > \alpha, \widehat{y_i} = -1 \; otherwise$

Optimizing for Precision vs Recall

- An optimistic model (one that predicts mostly positive) has **high recall, low precision**
- A pessimistic model (one that predicts mostly negative) has **high precision, low recall**
- The right option will depend on your problem!

Unbalanced Data

- What if your data is unbalanced (far more negative examples than positive, or vice versa)?
- This may cause poor **recall on the minority class**
  - Recall: of the samples that are actually the minority class, how many did the model predict as the minority class?
- One way to manage an unbalanced dataset is by **weighting** the loss function more heavily for the minority class
- Example for logistic regression

$$\hat{w} = argmin - \left(\alpha_{+1} \sum_{i=1:y_i=+1}^{n} \ln\left(\frac{1}{1+e^{-w^T h(x)}}\right) + \alpha_{-1} \sum_{i=1:y_i=-1}^{n} \ln\left(\frac{1}{1+e^{w^T h(x)}}\right) \right)$$

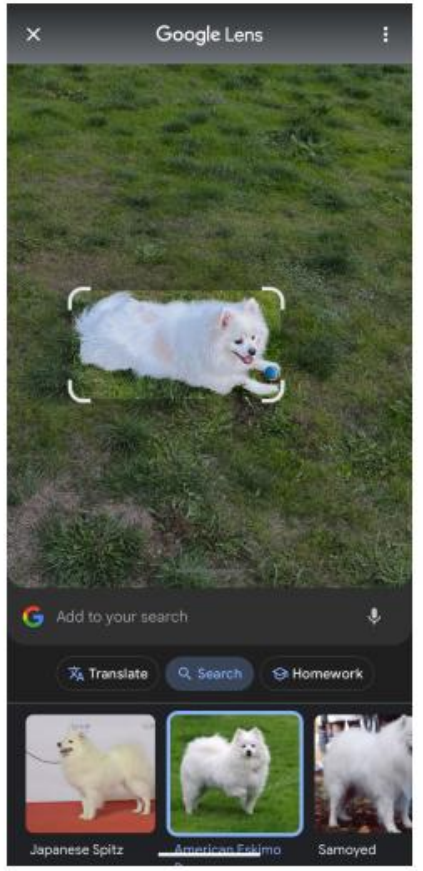
Break

Get some water, stretch your legs, chat, be back in 10

Multinomial Classification

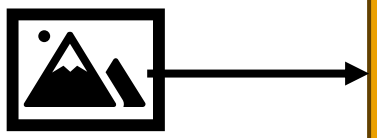
One-vs-all Classification

- We've covered a lot of methods for **binary** classification (only two classes)
- What do we do if we want to predict multiple classes?
- One basic idea is called **One-vs-All Classification**
- Let's examine this with Logistic Regression
- We have k classes
  - Train a binary classifier to predict +1 or -1
  - +1: $\hat{y} = class\ i$
  - -1: $\hat{y} \neq class\ i$



One-vs-all Classification

- Question: how many classifiers do we need?



image

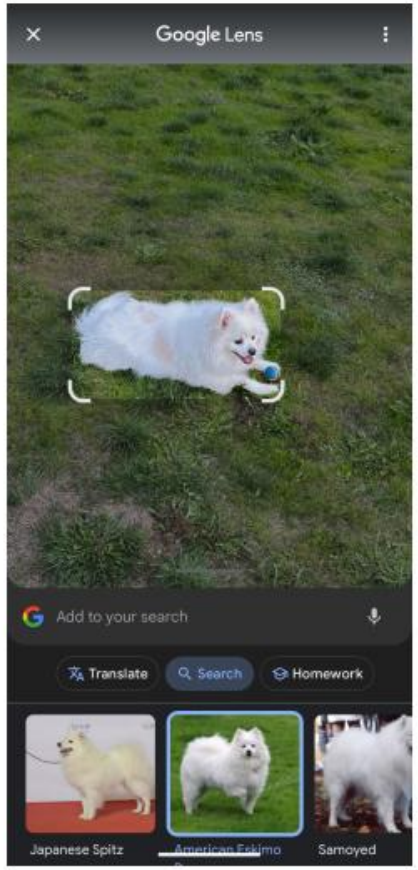
Logistic Regression

P(dog) = 0.8

P(not dog) = 0.2

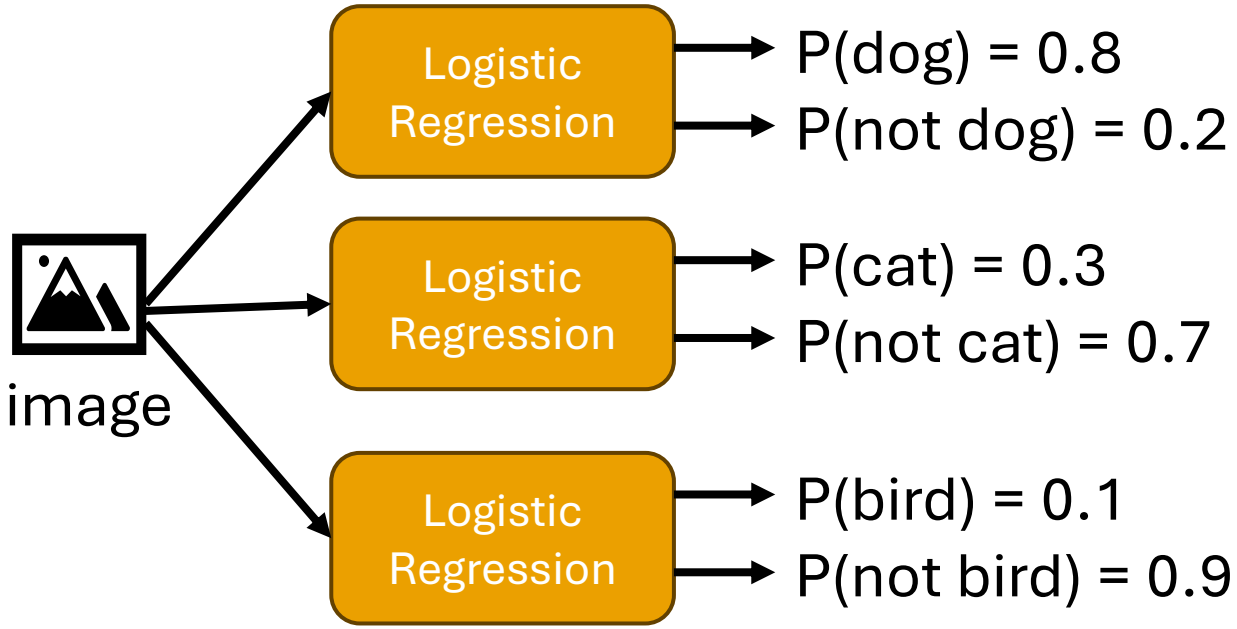
x

y

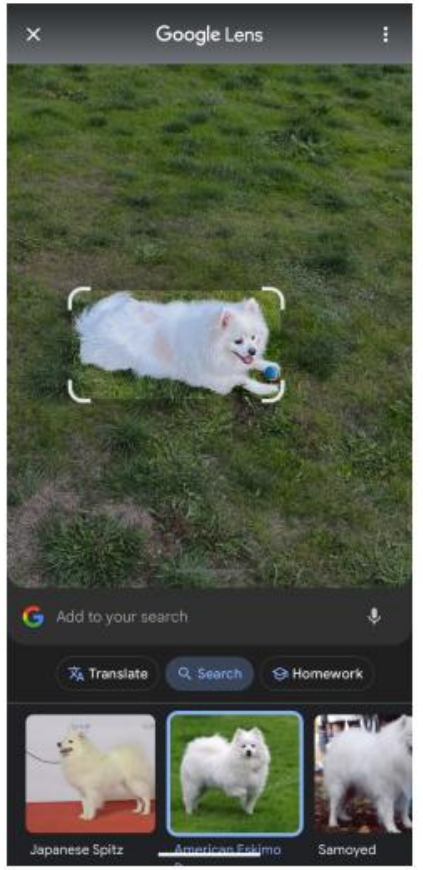


One-vs-all Classification

- Question: how many classifiers do we need?
- Answer: k classifiers, one for each class
- Example: predict dog, cat, or bird from an image



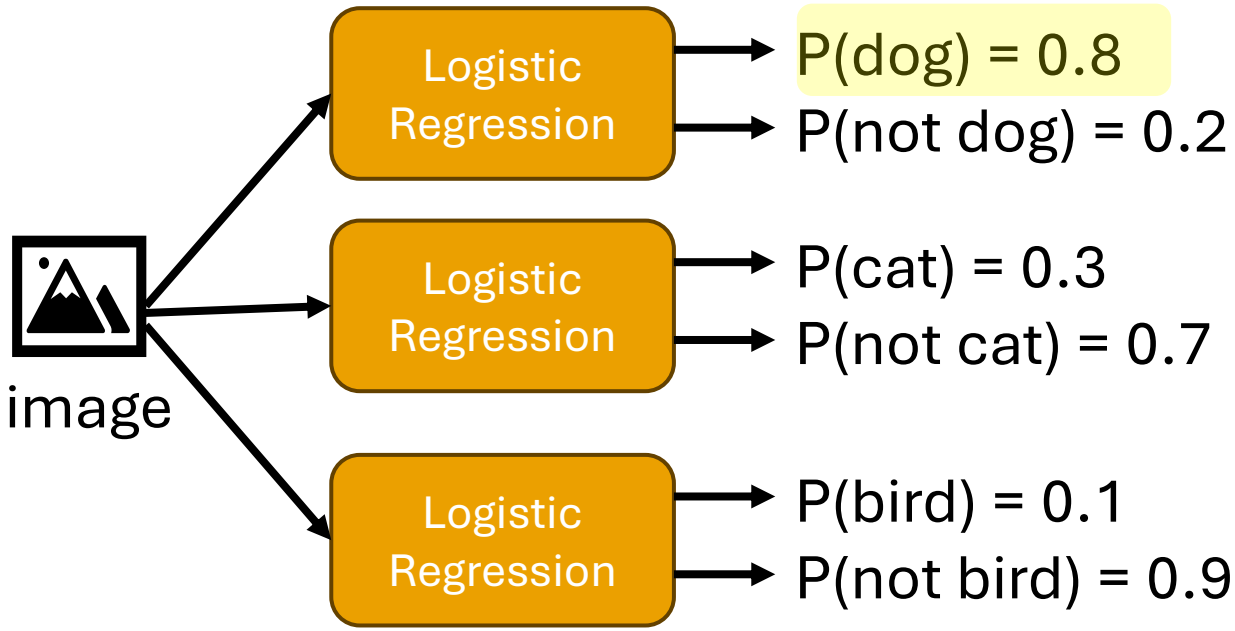
x



y

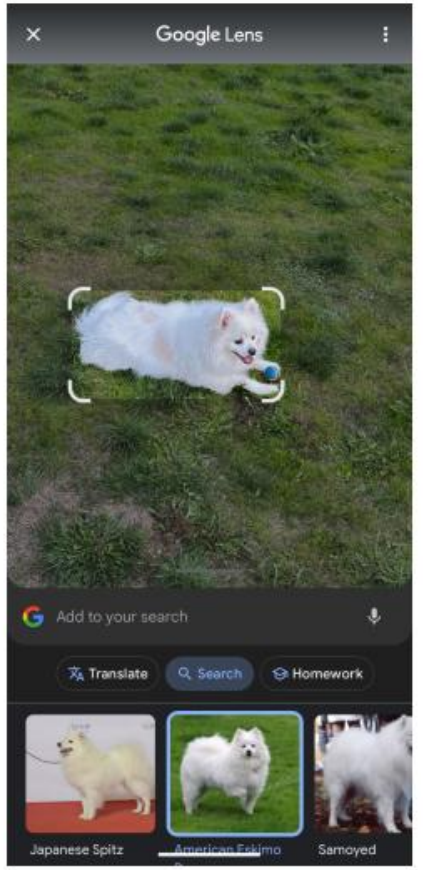
One-vs-all Classification

- The one-vs-all classifier outputs the class with the highest score/probability



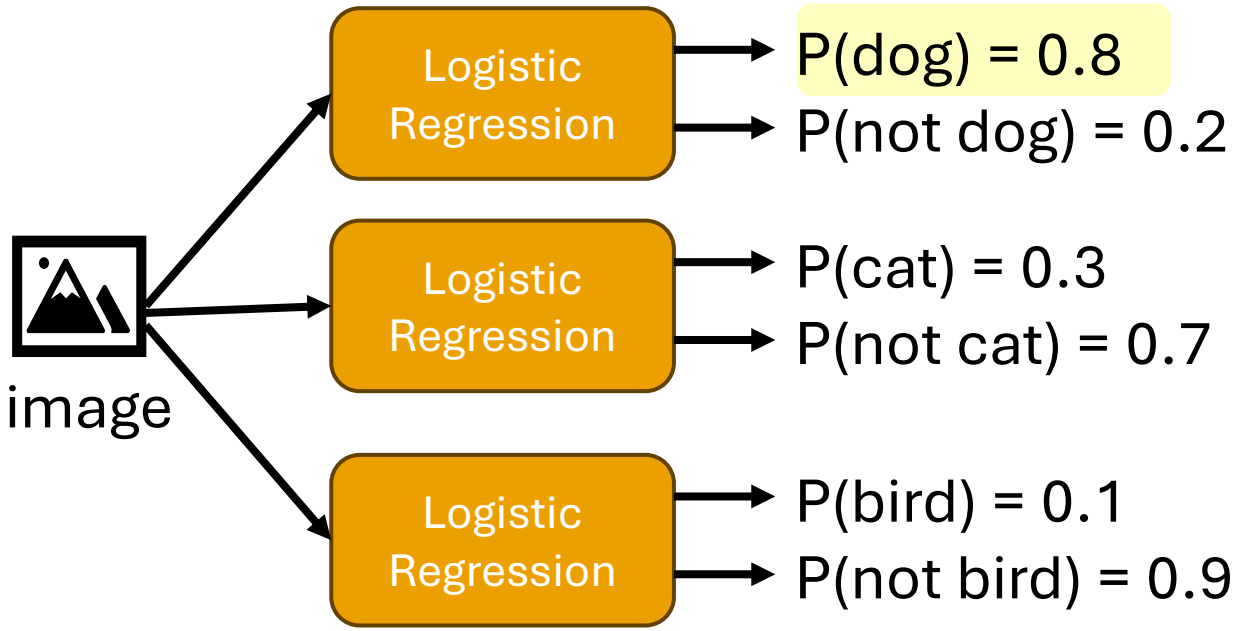
x

y



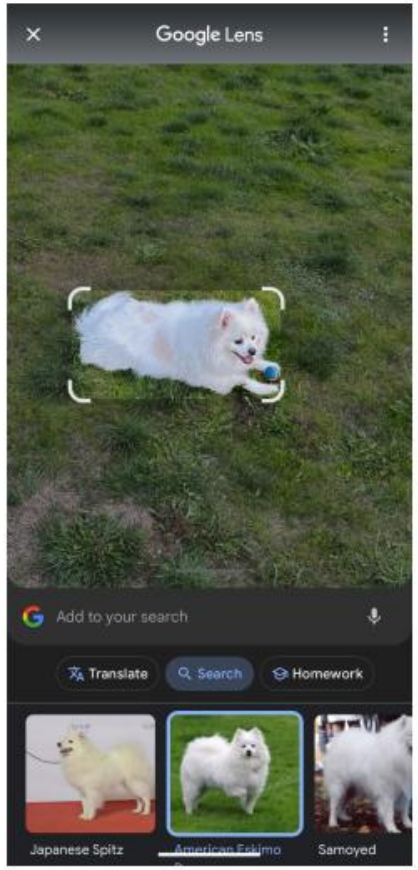
One-vs-all Classification

- Discuss: what downsides do you potentially see in this approach?



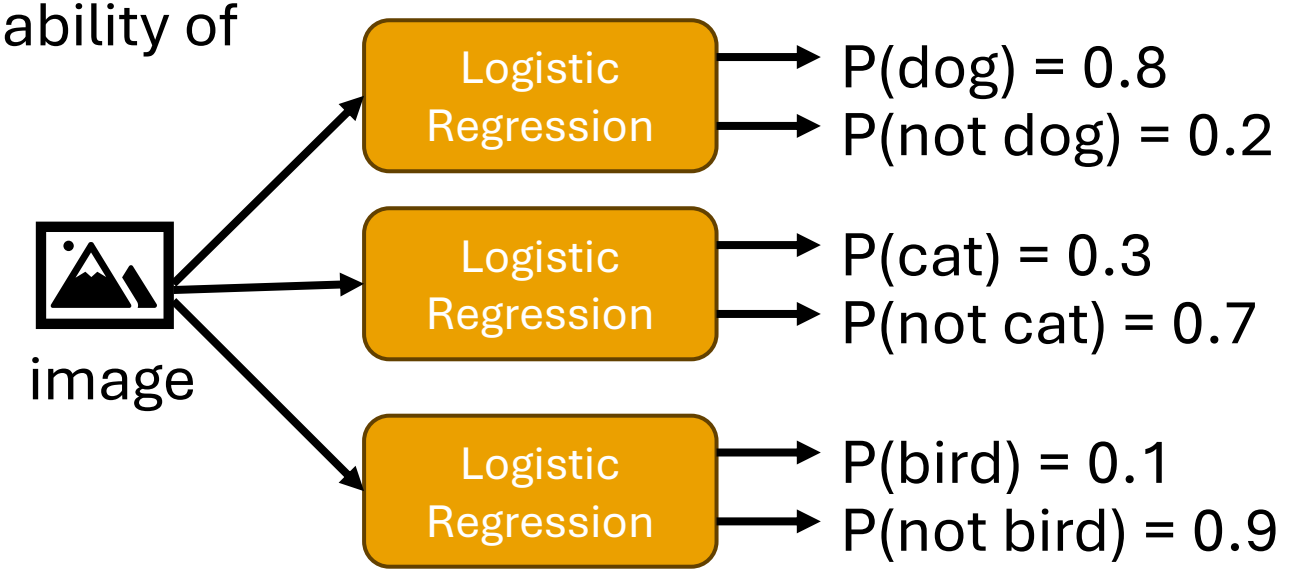
x

y



One-vs-all Classification

- Discuss: what downsides do you potentially see in this approach?
- Some downsides:
  - We have to train k classifiers (computationally expensive)
  - Probabilities can tie, and we have to choose how to break that tie
  - Probabilities aren't normalized: for this example, our model says the probability of the image being a dog is 0.8, and the probability of the image being a cat is 0.3. The probabilities for each class won't sum to 1.



Softmax Logistic Regression

- Logistic Regression lends itself well to multinomial classification
- We can use a type of Logistic Regression called **Softmax**
- However, we'll need to format our classification problem slightly differently
- Now, our $y_i \in \{c_1, c_2, \dots, c_k\}$
- Question: how did we deal with categorical data last week?

Softmax Logistic Regression

- Logistic Regression lends itself well to multinomial classification
- We can use a type of Logistic Regression called **Softmax**
- However, we'll need to format our classification problem slightly differently
- Now, our $y_i \in \{c_1, c_2, \dots, c_k\}$
- Question: how did we deal with categorical data last week?
- Answer: one-hot encoding!
- Instead of $y_i \in \{-1, 1\}$, we'll have $y_i = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}$ when $y_i = c_1$, and so on

Softmax Logistic Regression

- $y_i = \begin{bmatrix} 1 \\ 0 \\ 0 \\ \dots \\ 0 \end{bmatrix}$ when $y_i = c_1$, and so on
- Instead of predicting $\widehat{y_i} \in [-1,1]$, our new models will predict $\widehat{y_i} \in \mathbb{R}^k$
- We need our outputs to be k-dimensional instead of 1-dimensional
- We know that $x_i \in \mathbb{R}^d$ (d-dimensional vectors), and $\widehat{y_i} \in \mathbb{R}^k$ (k-dimensional vectors)
- So instead of our weights being $w \in \mathbb{R}^d$ (d-dimensional vectors), they must be $w \in \mathbb{R}^{dxk}$
- $\mathbb{R}^{dxk^T}\mathbb{R}^d = \mathbb{R}^k$

Softmax Logistic Regression

- Written out:

$$f(x_i) = \begin{bmatrix} f_1(x_i) \\ f_2(x_i) \\ \vdots \\ f_k(x_i) \end{bmatrix} = \underbrace{\begin{bmatrix} w_{1,0} & w_{1,1} & w_{1,2} & \cdots \\ w_{2,0} & w_{2,1} & w_{2,2} & \cdots \\ \vdots & & & \\ w_{k,0} & w_{k,1} & w_{k,2} & \cdots \end{bmatrix}}_{w^T} \underbrace{\begin{bmatrix} 1 \\ x_i[1] \\ \vdots \\ x_i[d] \end{bmatrix}}_{x_i} = \begin{bmatrix} w_{1,0} + w_{1,1}x_i[1] + w_{1,2}x_i[2] + \cdots \\ w_{2,0} + w_{2,1}x_i[1] + w_{2,2}x_i[2] + \cdots \\ \vdots \\ w_{k,0} + w_{k,1}x_i[1] + w_{k,2}x_i[2] + \cdots \end{bmatrix}$$

$$w = \begin{bmatrix} w[:,1] & w[:,2] & \cdots & w[:,k] \end{bmatrix}$$

Softmax Logistic Regression

- Previously, we had
  - $P(y_i = +1|x_i) = \frac{1}{1+e^{-\hat{w}^T x_i}} = \frac{e^{\hat{w}^T x_i}}{1+e^{\hat{w}^T x_i}}$
  - $P(y_i = -1|x_i) = \frac{1}{1+e^{\hat{w}^T x_i}} = \frac{e^{\hat{w}^T x_i}}{1+e^{-\hat{w}^T x_i}}$
- These probabilities are normalized ($P(y = +1|x) + P(y = -1|x) = 1$) for two classes
- For k classes, we'll have to change these probabilities to keep them normalized
  - $P(y_i = c_1|x_i) = \frac{e^{\hat{w}[:,1]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$
  - …
  - $P(y_i = c_k|x_i) = \frac{e^{\hat{w}[:,k]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$

Softmax Logistic Regression

- $P(y_i = c_1|x_i) = \frac{e^{\hat{w}[:,1]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$
- …
- $P(y_i = c_k|x_i) = \frac{e^{\hat{w}[:,k]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$

- $P(y_i = +1|x_i) = \frac{1}{1+e^{-\hat{w}^T x_i}} = \frac{e^{\hat{w}^T x_i}}{1+e^{\hat{w}^T x_i}}$
- $P(y_i = -1|x_i) = \frac{1}{1+e^{\hat{w}^T x_i}} = \frac{e^{\hat{w}^T x_i}}{1+e^{-\hat{w}^T x_i}}$

- Is this a completely different formula from binary Logistic Regression?
- No! Remember in multinomial regression we have $w = [w[:,1], w[:,2] \ldots w[:,k]]$
  - For the binary case, we can simplify by setting $w[:,1] = 0$, because we only need to find one probability (since $P(y_i = -1|x_i) = 1 - P(y_i = +1|x_i)$)
  - $P(y_i = c_1 = -1|x_i) = \frac{e^0}{e^0 + e^{\hat{w}[:,2]^T x_i}} = \frac{1}{1+e^{\hat{w}[:,2]^T x_i}}$
  - $P(y_i = c_2 = +1|x_i) = \frac{e^{\hat{w}[:,2]^T x_i}}{e^0 + e^{\hat{w}[:,2]^T x_i}} = \frac{e^{\hat{w}[:,2]^T x_i}}{1+e^{\hat{w}[:,2]^T x_i}}$

Softmax Logistic Regression

- $P(y_i = c_1 | x_i) = \frac{e^{\hat{w}[:,1]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$
- …
- $P(y_i = c_k | x_i) = \frac{e^{\hat{w}[:,k]^T x_i}}{e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}}$

- Because the numerator of $e^{\hat{w}[:,j \in [1,2,\ldots k]^T x_i}$ is divided by $e^{\hat{w}[:,1]^T x_i} + e^{\hat{w}[:,2]^T x_i} \ldots + e^{\hat{w}[:,k]^T x_i}$, the sum of all probabilities is 1
- Now we can interpret probabilities for different classes!
- But we're not done yet…

Softmax Logistic Regression

- Our loss function (Binary Cross-Entropy Loss) is written for binary values

$$-\frac{1}{n}\left(\sum_{i=1:y_i=+1}^{n}\ln\left(\frac{1}{1+e^{-w^T h(x)}}\right)+\sum_{i=1:y_i=-1}^{n}\ln\left(\frac{1}{1+e^{w^T h(x)}}\right)\right)$$

- This is just an application of a **Maximum Likelihood Estimator**, flipped to be negative for gradient descent

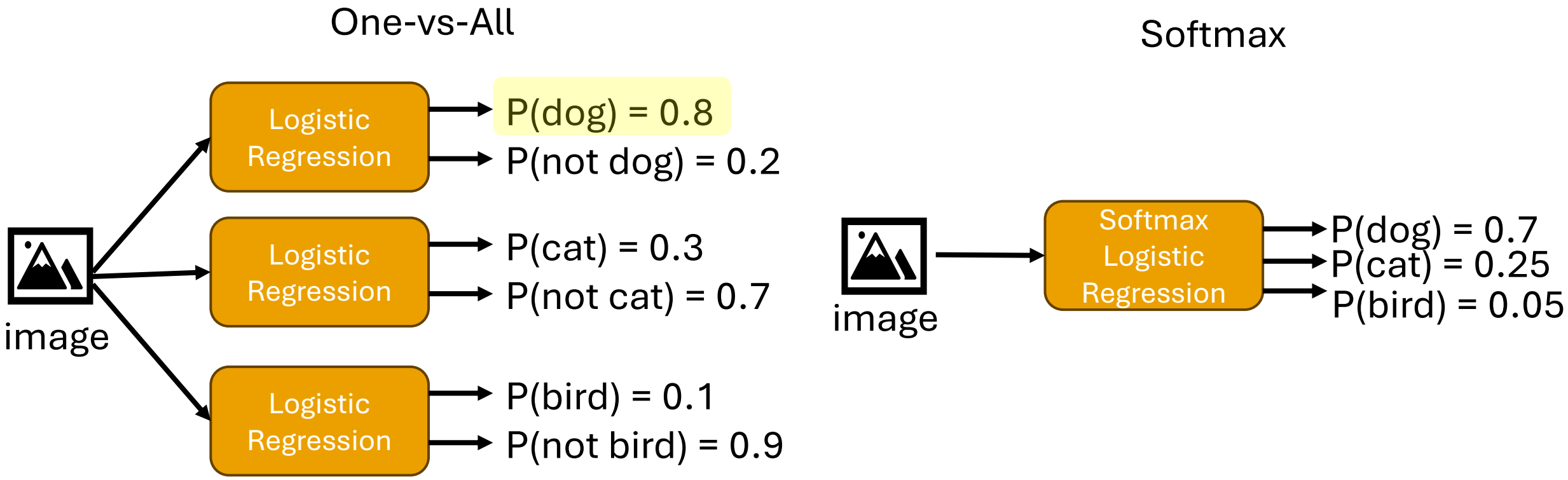
$$\max_{w}\frac{1}{n}\sum_{i=1}^{n}\ln(P(y_i|x_i))$$

- Now that we have a formula for $P(y_i|x_i)$ for Softmax Logistic Regression, we can plug it in

$$\min_{w\in\mathbb{R}^{dxk}}-\frac{1}{n}\sum_{i=1}^{n}\sum_{j=1}^{k}\mathbf{1}\{\mathrm{y_i}=\mathrm{c_j}\}\ln(\frac{e^{\widehat{w}[:,j]^T x_i}}{\sum_{m=1}^{k}e^{\widehat{w}[:,m]^T x_i}})$$

Softmax Logistic Regression

- Now, we can actually interpret our probabilities!



Metrics for Multinomial Classification

- Some of our previous metrics were defined for binary classification
- What about multinomial?
- Example: multinomial confusion matrix

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	100	25	31
Class 2	35	125	24
Class 3	2	18	340

Metrics for Multinomial Classification

- We can't refer to just False Negatives (FN), False Positives (FP), True Positives (TP), and True Negatives (TN) anymore
- However, we can refer to FN, FP, TP and TN **per class**

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	100	25	31
Class 2	35	125	24
Class 3	2	18	340

Metrics for Multinomial Classification

- We can't refer to just False Negatives (FN), False Positives (FP), True Positives (TP), and True Negatives (TN) anymore
- However, we can refer to FN, FP, TP and TN **per class**

Bolded: TP for Class 1 (predicted class 1, was class 1)

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	**100**	25	31
Class 2	35	125	24
Class 3	2	18	340

Metrics for Multinomial Classification

- We can't refer to just False Negatives (FN), False Positives (FP), True Positives (TP), and True Negatives (TN) anymore
- However, we can refer to FN, FP, TP and TN **per class**

Bolded: TN for Class 1 (predicted not class 1, was not class 1)

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	100	25	31
Class 2	35	**125**	**24**
Class 3	2	**18**	**340**

Metrics for Multinomial Classification

- We can't refer to just False Negatives (FN), False Positives (FP), True Positives (TP), and True Negatives (TN) anymore
- However, we can refer to FN, FP, TP and TN **per class**

Bolded: FP for Class 1 (predicted class 1, was not class 1)

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	100	25	31
Class 2	**35**	125	24
Class 3	**2**	18	340

Metrics for Multinomial Classification

- We can't refer to just False Negatives (FN), False Positives (FP), True Positives (TP), and True Negatives (TN) anymore
- However, we can refer to FN, FP, TP and TN **per class**

Bolded: FN for Class 1 (predicted not class 1, was class 1)

Predicted Label

True Label	Class 1	Class 2	Class 3
Class 1	100	**25**	**31**
Class 2	35	125	24
Class 3	2	18	340

Other metrics besides accuracy

- Now we can represent multinomial accuracy as $\frac{correctly\ classified}{all\ classified} = \frac{\sum_i^k TP_i+TN_i}{\sum_{i=1}^k TP_i+TN_i+FP_i+FN_i}$
- We can also measure our true positive rate (**recall) for class i**: $\frac{TP_i}{TP_i+FN_i}$
- **Precision**, all of the positive classifications that were correct **for class i**: $\frac{TP_i}{TP_i+FP_i}$
- **F1 score for class i**: $2 * \frac{Precision_i * Recall_i}{Precision_i+Recall_i}$
- Scores across classes are often averaged (or weighted averaged) to return a single score

Some pros and cons of presented classification methods

- Perceptron
  - Easy to implement, efficient
  - Can only solve problems with linearly separable data
- SVM
  - Good at handling high-dimensional data (like images)
  - With extensions, can work well for non-linear data
  - Still works when the number of features is greater than the dataset size
  - Can be slow with large datasets, tricky to tune
  - Memory-intensive for large datasets (have to store support vectors)
- Logistic Regression
  - Easily extended to multinomial classification, provides probabilities
  - Uses a linear function, making non-linearly separable data difficult
  - Doesn’t work well when the number of features is greater than the dataset size